A PRACTICAL PYTHON COMPANION

Python crash course

Learn through the code you already know.

Deep Learning with Python, third edition
Chapters 2 and 3

01 **Read the syntax**
Names, calls, containers, and execution

02 **Trace the data**
Shapes, slices, loops, and numerical operations

03 **Understand the objects**
Classes, state, gradients, and framework APIs

04 **Write it yourself**
Exercises, worked answers, and a complete lab

Personal study edition | 26 October 2026
Based on the third-edition companion notebooks in your workspace.

Your course map

Work from the foundations to the final lab, or use this clickable map to revisit a particular Python pattern. Exercise solutions follow each chapter-focused part.

Part I | Learn to read the code 4

1. How to use this course . . . 4
2. The execution environment is part of the lesson . . . 5
3. Names, values, types, and punctuation . . . 6
4. Imports, attributes, calls, and unpacking . . . 7
5. Read the first neural network as a Python program . . . 9

Part II | Python inside chapter 2 11

6. Read an array before operating on it . . . 11
7. Indexing, slices, and shared storage . . . 12
8. Functions, loops, and vectorized operations . . . 13
9. Broadcasting and matrix multiplication . . . 14
10. Reshape and transpose answer different questions . . . 15
11. Classes make state and behavior travel together . . . 16
12. Batches are a lesson in mutable state . . . 17
13. Gradients and the training loop: Python wiring . . . 19
14. Practice before reading the solutions . . . 21
15. Worked solutions for Part II . . . 22

Part III | Python inside chapter 3 24

16. Recognizing what belongs to Python . . . 24
17. Building the shared dataset without hidden steps . . . 25
18. Reading a plot as a Python expression . . . 26
19. TensorFlow: names, objects, and mutation . . . 26

20. with … as … controls a region of execution . . . 27

21. A complete TensorFlow step, read as Python . . . 28

22. Decorators: passing a function instead of calling it . . . 28

23. PyTorch: attributes that store gradients . . . 29

24. A corrected PyTorch step with explicit state . . . 30

25. Inheritance packages state with behavior . . . 31

26. JAX: familiar syntax, immutable arrays . . . 32

27. Random state can be implicit or explicit . . . 32

28. Functions can accept and return functions . . . 33

29. A complete JAX step returns its new state . . . 34

30. Keras: a callable class is a learning building block . . . 34

31. Lists of objects and training configuration . . . 35

32. Shuffle paired data with one permutation . . . 36

33. Return objects, dictionaries, and prediction batches . . . 37

34. Exercises: make the state changes explicit . . . 37

35. Worked solutions and reasoning . . . 38

Part IV | Read, repair, and rebuild **41**

36. Decode the notebook's backend helper . . . 41

37. A debugging routine you can reuse . . . 42

38. Repair cards for the supplied notebooks . . . 43

39. Final lab: build a tiny prediction pipeline . . . 43

40. A compact Python reference for these chapters . . . 47

Sources and verification **48**

41. What this course is based on . . . 48

42. Further reading for a specific question . . . 48

43. Verification and code license . . . 49

Part I | Learn to read the code

1. How to use this course

You have already read chapters 2 and 3 of *Deep Learning with Python*, third edition. This course revisits their code with a different question: what is Python doing at each step? The goal is to make you comfortable reading, changing, debugging, and writing that code yourself.

The source is the two third-edition companion notebooks in the main folder of your repository. The `first_edition` and `second_edition` folders were not used. This is an original teaching companion, not a reproduction of the book's prose. It covers the Python constructs that occur in these two notebooks, plus a few small extensions needed for practice. It does not claim to cover the entire Python language.

Each source locator gives a notebook chapter, a heading, and a zero-based cell index, counting both Markdown and code cells. For example, chapter 2 cell 6 is the seventh cell in the file. These are file positions, not the changing execution numbers shown beside notebook cells. Repeated code is explained once and connected to its later uses.

Read code actively. Before running a block, predict its result and the type and shape of each important value. Then run it. Change one thing, predict again, and explain the result in your own words. Typing a small example from memory is more useful than copying an entire training loop.

Example labels. Exact excerpts retain the source's logic. Adaptations use smaller inputs, clearer names, or explicit corrections. A block described as a continuation needs the objects defined immediately before it. Framework examples require the named framework; tiny Python and NumPy examples are the easiest place to begin.

A practical study route

Session	Work	You should be able to do this afterward
1	Part I: names, calls, containers, first model	Explain every bracket in a model definition
2	Part II: arrays and slicing	Predict shapes without running the cell
3	Part II: loops and tensor operations	Write and test a small array function
4	Part II: classes and batching	Trace an object's state across calls
5	Part II: manual training	Follow arguments, returns, and weight updates
6	Part III: TensorFlow and PyTorch	Distinguish assignment from mutation
7	Part III: JAX and custom Keras layers	Pass functions and explicit state correctly
8	Exercises, repairs, and the final lab	Rebuild a small program without copying

Allow roughly 45-90 minutes per session, depending on how much you type and experiment. Do the exercises before looking at their solutions. If a section feels easy, prove it by predicting the output of its examples.

2. The execution environment is part of the lesson

Source: both notebooks, setup cells 1-3; repository README, choosing a backend.

A notebook is a collection of cells sharing one running Python process, called a kernel. Variables survive between cell executions. If you execute a cell twice, it acts on the state left by the first execution. If you change an earlier function definition, later calls use the most recently executed definition.

The display order on screen is not proof of execution order. `NameError` often means the defining cell has not run. A mysterious wrong shape can mean a previous cell already reshaped the array. In chapter 2, `train_images` is initially three-dimensional, becomes two-dimensional after preprocessing, and later returns to three dimensions when MNIST is loaded again.

Use a fresh notebook for your exercises. For the original framework examples, use the environments described by the repository README. A useful check inside any notebook is:

PYTHON

```
import sys
print(sys.executable)
print(sys.version)
```

This tells you which Python process is actually running. Installing a package in one environment does not make it available to a notebook using a different environment.

Python syntax versus notebook syntax

OUTPUT / NOTEBOOK

```
!pip install keras keras-hub --upgrade -q
%%backend tensorflow
# @title
```

`!` asks IPython to run a shell command. It is not valid syntax in an ordinary `.py` file. The first line asks pip to install packages; `--upgrade` and `-q` are command-line options, not Python operators. `%%backend` is a custom cell magic registered by the notebook's helper. It is also not ordinary Python. The `# @title` line is a Python comment with additional meaning to Colab's interface. These setup conveniences are separate from the language you are learning.

The notebook package name `keras-hub` and Python import names are also different kinds of name. A hyphen is allowed in a package distribution name; in Python code, `keras-hub` would mean subtraction. You do not need KerasHub features for the core examples taught here.

Selecting the Keras backend

PYTHON

```
import os
os.environ["KERAS_BACKEND"] = "tensorflow"
import keras
```

`os` is a standard-library module. `os.environ` is a mapping of environment variable names to string values. The square brackets select a key; assignment stores the string `"tensorflow"` for that key. This configuration must happen before importing Keras in a fresh process. Changing the string after Keras has initialized does not switch the already-running backend. Restart the kernel when switching.

The local chapter 2 notebook selects TensorFlow; chapter 3 initially selects JAX. Importing `tensorflow` directly and selecting TensorFlow as Keras's backend are related but separate actions. The examples in chapter 3 also use each framework directly.

3. Names, values, types, and punctuation

Source: chapter 2, first example, cells 6-21; small examples below are teaching adaptations.

```python
epochs = 5
learning_rate = 1e-3
activation_name = "relu"
use_bias = True
activation = None
```

`=` binds a name to a value. The values above have types `int`, `float`, `str`, `bool`, and `NoneType`. `1e-3` means 0.001; it is one floating-point literal. `None` is the single Python object used to represent absence, such as an optional activation that was not provided. It is not the string `"None"`, zero, or an empty list.

Names are case-sensitive: `W` and `w` can refer to different objects. Underscores connect words in names such as `train_images`. Python does not require you to declare a variable's type, and a name can later be rebound to a different type. That freedom is useful, but reusing `model` for several unrelated functions and objects can make a long notebook harder to understand.

```python
pixels = 28 * 28
print(pixels, type(pixels))
print(5 / 2, 5 // 2, 5 % 2, 2 ** 3)
```

The first line computes 784 before assigning it. The second print displays values equivalent to `2.5 2 1 8`: true division, floor division, remainder, and exponentiation. `//` is introduced here for practice; the source batch class uses `math.ceil` instead. Multiplication and division bind more tightly than addition and subtraction. Parentheses make grouping explicit: `128 * (n + 1)` differs from `128 * n + 1`.

A punctuation decoder

Form	Meaning in this course
`name = value`	Bind a name; on an attribute or index, request an update
`a == b`	Compare values; arrays can produce an array of answers
`obj.attr`	Look up an attribute on an object
`f(x)`	Call a function or another callable object
`[a, b]`	Build a list
`(a, b)`	Build a tuple
`x[i]`	Select an item or array position
`x[start:stop]`	Slice, with stop excluded
`{"loss": values}`	Build a dictionary with a key and value
`def f(...):`	Start a function body, defined by indentation
`f"loss={loss:.2f}"`	Insert and format a value inside a string

The same character can have different roles. A colon in a slice separates bounds; a colon after `if` starts an indented block; a colon inside an f-string introduces a format. Learn the surrounding pattern rather than assigning one meaning to punctuation everywhere.

Lists, tuples, and dictionaries

PYTHON

```
units = [512, 10]
shape = (28, 28)
one_axis = (10,)
history = {"loss": [0.8, 0.4, 0.2]}
print(units[0], shape[-1], history["loss"][-1])
```

The output is `512 28 0.2`. Lists and tuples preserve order. Indexing starts at zero; `-1` selects the last item. Lists can have their contents changed, while a tuple's item references cannot be reassigned. The comma makes a one-element tuple: `(10,)` is a tuple, but `(10)` is just the integer 10 surrounded by grouping parentheses. An empty tuple is `()`; it represents a scalar array's shape later in the course.

A dictionary maps keys to values. Here the key is the string `"loss"` and its value is a list. Reading `history["loss"][-1]` means first retrieve that list, then retrieve its final item. A missing dictionary key raises `KeyError`. In `history.history`, the first `history` in the chapter 3 notebook is an object name, while the second is its attribute, which contains a dictionary.

Names point to objects

PYTHON

```
first = [1, 2]
second = first
second[0] = 99
print(first)              # [99, 2]
second = [7, 8]
print(first)              # [99, 2]
```

The assignment `second = first` does not copy a list. Both names initially refer to the same object, so changing an item through either name affects that object. Later, `second = [7, 8]` binds the name to a different object. It does not replace the object still referred to by `first`. This distinction explains much of the chapter 3 difference between rebinding a Python name and updating a framework variable.

An array's Python type and element `dtype` answer different questions. `type(x)` might be `numpy.ndarray`; `x.dtype` might be `float32`. The first describes the container object. The second describes how numerical elements are represented.

4. Imports, attributes, calls, and unpacking

PYTHON

```
import numpy as np
from keras import layers
from keras.datasets import mnist
```

`import numpy as np` imports a module and binds it to the local alias `np`. `as` does not change the library itself. `from keras import layers` brings a selected name into the current namespace; it does not create the separate name `keras`. If you later write `keras.Sequential`, you still need `import keras`.

Dots navigate attributes: `keras.datasets.mnist` names something inside something else. In `x.shape`, the attribute already holds a value, so you do not call it. In `x.reshape(...)`, the attribute is a method that you call. In `np.reshape(x, ...)`, a function in a module receives `x` explicitly. The API determines which form exists.

Trace a call from inside to outside

```python
import numpy as np
x = np.zeros((2, 3), dtype="float32")
print(x.shape)
```

First Python looks up `zeros` on the module `np`. It then evaluates the arguments from left to right, building the tuple `(2, 3)` and the string `"float32"`, and calls the function. `(2, 3)` is a positional argument: its location identifies the parameter. `dtype="float32"` is a keyword argument: its name identifies the parameter. The resulting array is bound to `x`. Finally, `print` receives `x.shape`, displaying `(2, 3)`.

The `=` in `dtype="float32"` supplies a named argument. It does not create a variable named `dtype` in your surrounding scope. Ordinary positional arguments come before keyword arguments. A parameter is a name in a function definition; an argument is the actual value supplied in a call.

Unpack the data structure, not just the words

Exact source pattern, chapter 2 cell 6:

```python
(train_images, train_labels), (test_images, test_labels) = (
    mnist.load_data()
)
```

`mnist.load_data()` returns a pair, and each element of that pair is another pair. Python matches the structure on the left to the returned structure on the right. It assigns the four contained arrays to four names. Parentheses around the right side merely let the expression continue across lines.

This tiny stand-alone example has the same structure:

```python
(train_x, train_y), (test_x, test_y) = (("A", "B"), ("C", "D"))
print(train_x, train_y, test_x, test_y)
```

It prints `A B C D`. A mismatched number of values raises `ValueError`; unpacking does not guess which value belongs to which name. `test_loss, test_acc = model.evaluate(...)` is the same idea with one level of nesting. The number of returned results depends on the configured metrics.

5. Read the first neural network as a Python program

Source: chapter 2, "A first look at a neural network", cells 13-21. The blocks below retain the source's model and data flow, with extra explanatory comments. They require Keras, a selected backend, and the MNIST data loaded in the preceding section. A first dataset load may download MNIST.

PYTHON

```
import keras
from keras import layers

model = keras.Sequential(
    [
        layers.Dense(512, activation="relu"),
        layers.Dense(10, activation="softmax"),
    ]
)
```

Read from the inside outward. Calling `layers.Dense` constructs one layer object. Calling it again constructs a second object. Square brackets put the two objects into a list, in order. `keras.Sequential` receives that list and constructs a model object. The name `model` then refers to that object.

The strings `"relu"` and `"softmax"` are values the built-in layer knows how to interpret. Python itself has no special activation syntax. A custom class must explicitly implement any string-to-function conversion it needs; this matters in the chapter 3 `SimpleDense` example.

The final comma after the second layer is legal and makes editing easier. Open brackets allow an expression to continue across physical lines. The indentation here makes nesting readable; indentation after `def`, `if`, `for`, `class`, and `with` additionally determines program structure.

PYTHON

```
model.compile(
    optimizer="adam",
    loss="sparse_categorical_crossentropy",
    metrics=["accuracy"],
)
```

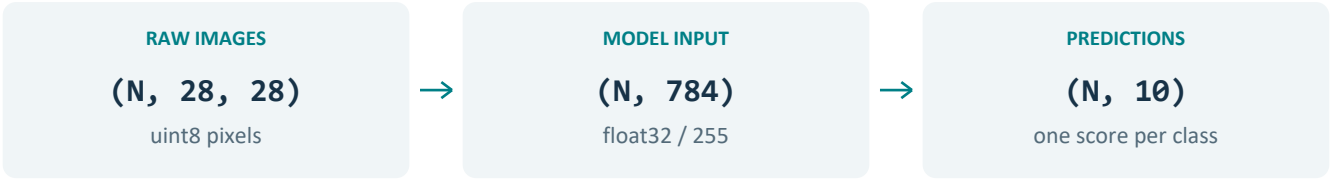
This calls a method on the existing model. All three arguments are keyword arguments. `metrics` receives a list containing one string, not a single string. The method configures how training and evaluation work. In this context Keras's `compile` method configures the model; it is distinct from the Python built-in `compile` and the framework decorators discussed later.

PYTHON

```
train_images = train_images.reshape((60000, 28 * 28))
train_images = train_images.astype("float32") / 255
test_images = test_images.reshape((10000, 28 * 28))
test_images = test_images.astype("float32") / 255
```

The right side is evaluated before the left name is rebound. `28 * 28` becomes 784; the shape tuple becomes `(60000, 784)`. `reshape` changes the arrangement into rows of pixels. `astype` converts element representation. Dividing by 255 performs element-wise normalization; it is not a Python loop. The labels remain class integers, so they are not divided by 255.

Later examples replace fixed sample counts with `len(images)` so they also work on small batches. Reshape preserves the number of elements. It does not resize images or invent pixels. Part II explains the shapes and memory behavior carefully.



N is the number of examples. Track this axis through every step.

PYTHON

```
history = model.fit(
    train_images, train_labels, epochs=5, batch_size=128
)
test_digits = test_images[0:10]
predictions = model.predict(test_digits)
predicted_class = predictions[0].argmax()
test_loss, test_acc = model.evaluate(test_images, test_labels)
print(f"test_acc: {test_acc:.3f}")
```

This adaptation saves the `History` returned by `fit` and names the predicted class. The first two arguments to `fit` are positional; the remaining arguments are named. A slice `0:10` selects ten examples, indices 0 through 9. Predictions have shape `(10, 10)` here: ten examples, each with ten class scores. `predictions[0]` selects one row, and `.argmax()` returns the index of its largest score. The source's `predictions[0][7]` instead selects the score for class 7; it does not find the predicted class.

The final line is an f-string. Braces contain a Python expression. `:.3f` asks for a fixed-point number with three digits after the decimal point. It changes the displayed string, not the stored accuracy. The actual scores and accuracy depend on training; do not memorize an exact result.

Your first prediction checks

1. In `layers.Dense(512, activation="relu")`, which argument is positional and which is named?
2. What is the difference between `(784)` and `(784,)`?
3. Does `model.compile(...)` create the variable `optimizer` in your notebook?
4. If `predictions.shape` is `(10, 10)`, what is the shape of `predictions[0]`?
5. Why do `test_images[0:10]` and `test_images[0]` have different ranks?

Answers. 1: `512` is positional; `activation` is named. 2: the first is an integer; the second is a one-element tuple. 3: no, it passes a keyword argument to a method. 4: `(10,)`. 5: a slice keeps the selected axis, while an integer index selects one item and removes that axis.

Part II | Python inside chapter 2

The chapter turns numerical ideas into executable Python. Our goal here is to read every important construct in that code, predict its behavior, and then change it confidently. The examples below use the repository's third-edition `chapter02_mathematical-building-blocks.ipynb`. A cell number is its zero-based position in the notebook file, including Markdown cells; it is not the notebook's changing execution count. "Exact excerpt" means copied code; "teaching adaptation" means modified or newly written code that exposes the same idea.

For the numerical examples in this part, run `import numpy as np` first. TensorFlow/Keras examples additionally need the TensorFlow backend selected before importing Keras. They are identified separately. Use small arrays while learning: they let you check your interpretation without training a model.

6. Read an array before operating on it

Notebook locator: "Data representations for neural networks," cells 24-38.

A NumPy array is one object containing numerical data plus metadata about its organization. `shape`, `ndim`, and `dtype` are attributes, so you read them without parentheses. By contrast, `reshape(...)` is a method: parentheses call it.

Teaching adaptation; dependency: NumPy.

PYTHON

```
scalar = np.array(12)
vector = np.array([12, 3, 6, 14, 7])
matrix = np.array([[1, 2, 3], [4, 5, 6]])
print(scalar.shape, scalar.ndim)
print(vector.shape, vector.ndim)
print(matrix.shape, matrix.ndim)
```

Expected output:

OUTPUT / NOTEBOOK

```
() 0
(5,) 1
(2, 3) 2
```

`np.array(...)` constructs an array. The nested brackets in the matrix describe two rows of three elements each. The scalar has no axes, so its shape is the empty tuple `()`. The vector's `(5,)` has a comma because this is a one-element tuple. `(5)` would simply be the integer `5` in parentheses. Tensor rank means number of axes here; it is not the linear-algebra notion of matrix rank.

An axis is a direction you can index. In `(60000, 28, 28)`, axis 0 selects an image, axis 1 a row, and axis 2 a column. The total number of values is the product of the shape entries. A rank-3 array need not represent three-dimensional physical space: the first axis could mean examples. Python and NumPy do not know the meaning of these axes; your program supplies it.

`len(array)` asks for the length along its first axis. For that image array it returns `60000`, not the number of pixels. `array.size` counts all elements, and `array.ndim` counts axes. `len(scalar)` fails because a rank-0 array has no first axis. This distinction prevents many incorrect loops.

The original MNIST images have dtype `uint8`: unsigned eight-bit integers ranging from 0 to 255. A dtype is the storage and arithmetic type of every element in the array; it is different from the Python type of the array object itself. `type(train_images)` is an array class, while `train_images.dtype` describes its elements. A list can freely mix objects; a regular numerical array uses a common dtype.

Notebook locator: "A first look at a neural network," cell 15, repeated in cell 90.

Exact excerpt:

```python
train_images = train_images.reshape((60000, 28 * 28))
train_images = train_images.astype("float32") / 255
```

First, Python calculates `28 * 28`. The reshape then groups the existing values into 60,000 rows of 784 features. Next, `astype` creates a floating-point conversion; division scales pixel values. The assignment rebinds `train_images` to the result. It does not somehow change every earlier variable that referred to the original array. The code uses a fixed dataset size; a reusable version is `images.reshape((len(images), -1))`, where `-1` asks NumPy to infer one dimension.

7. Indexing, slices, and shared storage

Before slicing batches, cell 39 displays one original MNIST image:

```python
import matplotlib.pyplot as plt

digit = train_images[4]
plt.imshow(digit, cmap=plt.cm.binary)
plt.show()
```

This requires Matplotlib and the original image array with shape `(60000, 28, 28)`, before flattening. `digit` has shape `(28, 28)`. `plt.cm.binary` retrieves a colormap object through two attribute lookups; `cmap=` passes that object as a keyword argument. It is not a string or a function call. `plt.imshow` draws the array as an image, and `plt.show()` displays the figure. Cell 40 reads `train_labels[4]`, using the same index to inspect its label. The index pairs an image with its answer; it is not itself the digit's class.

Notebook locator: "Manipulating tensors in NumPy," cells 42-46; "The notion of data batches," cells 48-50.

Python positions begin at zero. `images[4]` selects the fifth image and removes the example axis. `images[4:5]` selects a slice containing one image and preserves that axis. A slice `start:stop` includes `start` but excludes `stop`. This makes its length easy to calculate: `100 - 10 = 90` when both positions are within the array.

The chapter's three expressions `train_images[10:100]`, `train_images[10:100, :, :]`, and `train_images[10:100, 0:28, 0:28]` select the same region of the original MNIST array. A bare `:` means all entries on that axis. Commas separate axes inside a single indexing operation.

Teaching adaptation; dependency: NumPy.

```python
images = np.arange(3 * 4 * 4).reshape((3, 4, 4))
print(images[0].shape)
print(images[0:1].shape)
print(images[:, 2:, 2:].shape)
print(images[:, 1:-1, 1:-1].shape)
```

Expected shapes are `(4, 4)`, `(1, 4, 4)`, `(3, 2, 2)`, and `(3, 2, 2)`. `np.arange` creates consecutive values; here the values help you track where each slice came from. Negative indices count from the end. A stop of `-1` excludes the last position; `7:-7` in a 28-pixel dimension retains positions 7 through 20, producing length 14. Omitting the stop, as in `14:`, includes the final element.

You can also write `start:stop:step`. `vector[::2]` takes every second value, and `vector[::-1]` reverses the order through a view. Unlike indexing a single missing position, an oversized slice stop is normally clipped to the array boundary. That is why the last batch can be shorter without special slicing code.

The crucial hidden detail is that basic NumPy slicing usually returns a view: a different array object sharing the original storage. Changing a view can change its source. Assignment of a name never promises an independent copy.

Teaching adaptation; expected final array: `[0, 99, 2, 3, 4]`.

```python
original = np.arange(5)
view = original[1:4]
view[0] = 99
independent = original[1:4].copy()
independent[0] = -1
print(original)
```

`view[0]` maps to `original[1]`. Calling `.copy()` creates separate storage. The chapter uses this deliberately in its naïve functions: they modify a working copy and return it, leaving the caller's array intact. Advanced indexing with an integer list or a Boolean mask generally produces a copy instead; do not extend the basic-slice rule to all indexing. Use `np.shares_memory(a, b)` to investigate an actual pair.

For batches, the expression `images[size * n : size * (n + 1)]` selects batch number `n`, with `n=0` selecting the first batch. In cell 50, `n=3` and size 128 select positions 384 through 511. The arithmetic belongs inside the brackets because it calculates slice boundaries before indexing occurs.

8. Functions, loops, and vectorized operations

Notebook locator: "Element-wise operations," cells 58-61.

Exact excerpt, cell 58:

```python
def naive_relu(x):
    assert len(x.shape) == 2
    x = x.copy()
    for i in range(x.shape[0]):
        for j in range(x.shape[1]):
            x[i, j] = max(x[i, j], 0)
    return x
```

`def` binds a function object to the name `naive_relu`; its body runs only when called. The parameter `x` is a local name bound to the supplied object. `assert` checks a condition and raises `AssertionError` if it is false. Here, it documents the teaching assumption that the input is a matrix. Assertions can be disabled with optimized Python execution, so public input validation should raise an explicit exception instead.

After `x = x.copy()`, the local name points at a fresh array. `range(n)` produces integers from zero through `n - 1` as the loop requests them. The outer loop visits rows; the inner loop visits every column for each row. Indentation defines which statements belong to each loop. `return x` is outside both loops, so the function returns only after all elements have been processed. Put it inside a loop accidentally and you stop early.

Built-in `max(a, b)` chooses the greater of two scalar values here. NumPy's `np.maximum(array, 0)` instead compares element by element across an entire array. Do not substitute `max(array, 0)`: built-in scalar-oriented comparisons and array comparisons have different semantics.

In `naive_add`, cell 59, the second assertion is `x.shape == y.shape`; tuple equality gives one Boolean result. The indexed statement `x[i, j] += y[i, j]` adds into an element of the copy. For array objects, `x += y` generally

modifies existing storage, while `x = x + y` computes a result and rebinds the local name. Dtypes and broadcasting can also make an in-place operation stricter.

Teaching adaptation; dependency: NumPy and `naive_relu` above.

```python
values = np.array([[-2, 3], [0, -4]])
result = naive_relu(values)
print(result.tolist())
print(values.tolist())
assert np.array_equal(result, np.maximum(values, 0))
```

Expected lists are `[[0, 3], [0, 0]]` and `[[-2, 3], [0, -4]]`. `.tolist()` makes the printed structure familiar; it is not needed for computation. `np.array_equal` yields one Boolean. By contrast, `result == values` yields an array of elementwise Booleans. Python cannot generally decide whether such an array should count as one true-or-false condition.

The chapter times 1,000 repetitions with `time.time()`, saving a start time and subtracting it afterward. `_` in `for _ in range(1000)` is a conventional name for an unused loop variable; Python still assigns it normally. The chapter's `"Took: {0:.2f} s".format(elapsed)` inserts argument zero as a floating-point value with two decimal places. `f"Took: {elapsed:.2f} s"` expresses the same formatting directly.

For elapsed-time measurements prefer `time.perf_counter()`. Timing is an experiment, not a fixed expected output: machine load, initialization, and input sizes affect results. NumPy's vectorized operations typically avoid running a Python loop for every element. "Vectorized" describes how you express and execute the operation; it does not mean your input must be a rank-1 vector.

9. Broadcasting and matrix multiplication

Notebook locator: "Broadcasting," cells 63-67; "Tensor product," cells 69-73.

Broadcasting lets compatible shapes participate in elementwise operations without writing a loop or manually repeating values. Compare shape dimensions from right to left. Each pair must either match or include a size 1; missing leading dimensions act like size 1. Thus `(32, 10)` and `(10,)` combine to `(32, 10)`. A `(32,)` vector does not combine with `(32, 10)` merely because its length matches the first dimension: alignment starts at the right.

Teaching adaptation; expected rows: `[11, 22, 33]` and `[14, 25, 36]`.

```python
x = np.array([[1, 2, 3], [4, 5, 6]])
bias = np.array([10, 20, 30])
print(x + bias)
row = np.expand_dims(bias, axis=0)
repeated = np.tile(row, (2, 1))
assert row.shape == (1, 3)
assert np.array_equal(x + bias, x + repeated)
```

`expand_dims` inserts a size-one axis: `(3,)` becomes `(1, 3)` when `axis=0`. With `axis=1` it becomes `(3, 1)`, useful for different broadcasting problems. In `tile(row, (2, 1))`, repeat the row twice along axis 0 and once along axis 1. Tiling physically constructs repeated data; ordinary broadcasting does not require this duplicated array. The chapter illustrates the idea with tiling, but `x + bias` is the direct expression you usually want.

Cell 67 combines `(64, 3, 32, 10)` and `(32, 10)` using `np.maximum`. Think of the latter shape as `(1, 1, 32, 10)`: it can be compared against every corresponding block of the larger tensor. The custom function in cell 66 does the matrix/vector version by reading `y[j]` for every row `i`; there is no `i` in that lookup because every row shares the vector.

Elementwise multiplication and matrix multiplication are separate operations. `*` multiplies corresponding elements, with broadcasting. `@` invokes matrix multiplication rules. `np.matmul(x, y)` is the NumPy function form used alongside `x @ y` in cell 69. For two one-dimensional arrays it returns their dot product, a sum of products.

Teaching adaptation; dependency: NumPy.

```python
a = np.array([1, 2, 3])
b = np.array([10, 20, 30])
print(a * b)
print(a @ b)
matrix = np.array([[1, 2, 3], [4, 5, 6]])
print(matrix @ b)
```

Expected results are `[10, 40, 90]`, `140`, and `[140, 320]`. Matrix/vector multiplication turns shape `(rows, columns)` and `(columns,)` into `(rows,)`. Matrix/matrix multiplication turns `(m, k)` and `(k, n)` into `(m, n)`. The shared dimension is summed over. Elementwise multiplication has no such summation.

Cell 70 initializes `z = 0.0` before its loop and accumulates `z += x[i] * y[i]`. An accumulator must be initialized before use and outside the loop it summarizes. If `z = 0.0` were inside the loop, only the final product would remain. Cell 71 uses `np.zeros(x.shape[0])` to create one accumulator per output row. Its default dtype is floating point.

Cell 72 defines `naive_matrix_vector_product` a second time. This replaces the earlier binding of that function name; it does not overload the function based on argument types. The replacement calls `naive_vector_product(x[i, :], y)` for each row. Cell 73 extends this pattern: `x[i, :]` supplies a row, `y[:, j]` supplies a column, and their dot product becomes `z[i, j]`.

10. Reshape and transpose answer different questions

Notebook locator: "Tensor reshaping," cells 75-79.

Reshaping changes how an ordered collection of values is grouped into axes. The total element count must stay the same. Transposing changes axis order. These can produce the same output shape but different element arrangements.

Teaching adaptation; dependency: NumPy.

```python
x = np.array([[0., 1.], [2., 3.], [4., 5.]])
print(x.reshape((2, 3)))
print(x.T)
```

The reshape produces rows `[0, 1, 2]` and `[3, 4, 5]`. The transpose produces `[0, 2, 4]` and `[1, 3, 5]`. Both shapes are `(2, 3)`, so checking shape alone would not catch a confusion between these operations. The decimal points in `0.` and `1.` denote floating-point literals.

`x.T` is the convenient matrix-transpose attribute; `np.transpose(x)` is the chapter's function form. For higher-rank arrays, transpose can take an explicit axis permutation. The default reverses the axis order. A rank-1 array stays rank 1 after `.T`; turning a vector into a column requires an explicit size-one axis.

Reshape may return a view or need a copy depending on the layout. Do not rely on reshape to isolate your data, and do not assume every reshape is free. If independent storage matters, request a copy explicitly. Neither operation normalizes values or changes the meaning of labels; those are separate tasks.

11. Classes make state and behavior travel together

Notebook locator: "A simple Dense class," cell 96; "A simple Sequential class," cells 98-99.

`class NaiveDense:` defines a new Python class. Calling `NaiveDense(...)` creates an instance and initializes it through `__init__`. Python supplies the instance as the first method argument, conventionally named `self`. `self.W` belongs to that instance; an ordinary local variable such as `x` exists only in the method call. Two layer instances therefore retain different weights.

The following is a teaching adaptation of cell 96, compressed without changing its structure. Dependencies: Keras with a configured backend, `from keras import ops`.

PYTHON

```
class NaiveDense:
    def __init__(self, input_size, output_size, activation=None):
        self.activation = activation
        self.W = keras.Variable(
            shape=(input_size, output_size), initializer="uniform"
        )
        self.b = keras.Variable(shape=(output_size,), initializer="zeros")

    def __call__(self, inputs):
        x = ops.matmul(inputs, self.W) + self.b
        if self.activation is not None:
            x = self.activation(x)
        return x

    @property
    def weights(self):
        return [self.W, self.b]
```

The initializer strings tell Keras how to create numerical weights. Python handles argument passing and attribute assignment; Keras supplies numerical storage and trainable variables. `(output_size,)` is again a one-element shape tuple. `None` is a special singleton used here to mean "no activation." `is not None` checks identity against that singleton and is the normal Python idiom.

`activation=ops.relu` passes the function object so the layer can call it later. `activation=ops.relu(x)` would call it immediately and pass the result instead. The expression `self.activation(x)` is the later invocation. Functions are values: you can store them in variables, pass them as arguments, and retain them in attributes.

Defining `__call__` makes an instance callable. After `layer = NaiveDense(3, 2)`, `layer(inputs)` invokes its `__call__` method. Construction and prediction are different calls: `NaiveDense(...)` initializes state; `layer(...)` uses that state. For inputs shaped `(5, 3)`, the matrix multiplication produces `(5, 2)`, and the bias of shape `(2,)` broadcasts across five examples.

`@property` is decorator syntax. It transforms the `weights` method into an attribute-style interface. You write `layer.weights`, without parentheses; reading it executes the getter and returns a newly created Python list containing references to the two variables. The variables themselves are not copied. `layer.weights()` would try to call the returned list and fail.

Cell 98 stores a list of layers in `self.layers`. Its `for layer in self.layers` loop repeatedly replaces local `x` with `layer(x)`, passing each result into the next layer. It is function composition expressed with ordinary iteration. Its weights property starts with `weights = []` and executes `weights += layer.weights` for every layer.

Here is the complete source class, so you can assemble the manual model:

PYTHON

```
class NaiveSequential:
    def __init__(self, layers):
        self.layers = layers

    def __call__(self, inputs):
        x = inputs
        for layer in self.layers:
            x = layer(x)
        return x

    @property
    def weights(self):
        weights = []
        for layer in self.layers:
            weights += layer.weights
        return weights
```

For Python lists, `+=` extends the current list with the items from another iterable. It does not add corresponding numerical values. `weights.append(layer.weights)` would instead append an entire list as one item, producing nested lists. Two dense layers each expose two variables, so the flattened list has length four, matching `assert len(model.weights) == 4` in cell 99. Keras classes in real applications add automatic tracking and many conveniences; this plain class teaches the Python mechanics.

12. Batches are a lesson in mutable state

Notebook locator: "A batch generator," cell 101.

The chapter's `BatchGenerator` stores arrays, a batch size, an index initially equal to zero, and `num_batches = math.ceil(len(images) / batch_size)`. `math.ceil` rounds upward: 10 examples with batches of 4 require three batches. `/` performs division; the result here is passed to `ceil`. The integer-only alternative is `(count + size - 1) // size`, using floor division.

Each call to its ordinary `.next()` method slices both arrays from the current index, increases `self.index`, and returns `images, labels`. Returning comma-separated values packages them as a tuple; `images_batch, labels_batch = ...` unpacks that tuple. Changing `self.index` affects the next call because instance attributes persist.

Important distinction: calling a method named `.next()` does not make an object a Python iterator. Python's `next(obj)` and `for ... in obj` use special methods `__next__` and `__iter__`. The chapter's class does not define those. It also does not raise `StopIteration` after its final batch; additional `.next()` calls yield empty slices. Its training loop avoids that by calling it exactly `num_batches` times.

The source class is short enough to trace in full. Assume matching image and label counts and a positive batch size:

```python
import math

class BatchGenerator:
    def __init__(self, images, labels, batch_size=128):
        self.index = 0
        self.images = images
        self.labels = labels
        self.batch_size = batch_size
        self.num_batches = math.ceil(len(images) / batch_size)

    def next(self):
        images = self.images[self.index:self.index + self.batch_size]
        labels = self.labels[self.index:self.index + self.batch_size]
        self.index += self.batch_size
        return images, labels
```

The default `batch_size=128` is used when the caller omits that argument. Defaults are evaluated when the function definition executes, not on each call. An integer default is safe here; mutable defaults such as a list can accidentally share state across calls. `images` inside `next` is a local slice; `self.images` remains the full stored array. The similarly named objects have different scopes and roles.

This separate teaching adaptation demonstrates the iterator protocol; dependency: NumPy for the example arrays. It is not a replacement required by the chapter.

```python
class Batches:
    def __init__(self, images, labels, size=4):
        if len(images) != len(labels) or size <= 0:
            raise ValueError("Check lengths and positive batch size")
        self.images, self.labels = images, labels
        self.size, self.index = size, 0

    def __iter__(self):
        return self

    def __next__(self):
        if self.index >= len(self.images):
            raise StopIteration
        start = self.index
        self.index += self.size
        return (self.images[start:self.index],
                self.labels[start:self.index])
```

With `Batches(np.arange(10), np.arange(10))`, a loop receives sizes 4, 4, and 2. Once consumed, this particular iterator remains exhausted. Create a new instance for a new epoch. A reusable iterable and an iterator are related but different interfaces; returning `self` implements the simple single-pass iterator pattern.

13. Gradients and the training loop: Python wiring

Notebook locator: "The weight update step," cells 104-105; "Gradient computation," cells 107-108; "The full training loop," cells 110-111.

In cell 104, `learning_rate = 1e-3` is scientific notation for `0.001`. `zip(gradients, weights)` produces pairs, and `for g, w in ...` unpacks each pair. Pair order matters: every gradient must be paired with the variable it describes. Ordinary `zip` stops at the shorter input; it does not flag unequal lengths. For known lists, compare lengths first, or use `zip(..., strict=True)` on Python 3.10 or newer.

`w.assign(w - g * learning_rate)` updates the contents of a backend variable. Writing `w = w - g * learning_rate` would merely rebind the loop's local name to a result. It would not perform the intended variable update. Cell 105 defines `update_weights` again using an optimizer; subsequent calls resolve to this newer definition. That function accesses `optimizer` from its surrounding module/notebook namespace.

The notebook contains a small trap in cell 107: `x = tf.zeros(shape=())` creates a constant scalar tensor. A gradient tape automatically watches trainable TensorFlow variables, but it does not automatically watch this constant. Consequently, the original demonstration yields `None` for its gradient. Here is an explicitly corrected teaching version; dependency: TensorFlow.

```python
import tensorflow as tf

x = tf.zeros(shape=())
with tf.GradientTape() as tape:
    tape.watch(x)
    y = 2 * x + 3
gradient = tape.gradient(y, x)
print(float(gradient))
```

Expected result: `2.0`. `with` enters and exits a context manager, which here establishes recording for the indented operations. `as tape` binds the resulting tape object. Calling `.gradient` happens after the block in this example. `None` means a gradient was not available; it is not a numerical zero. Replacing the constant with a suitable `tf.Variable` is another way to make this example differentiable.

Cell 108 calls the model, obtains a vector of per-example losses, averages it to one scalar, differentiates that scalar with respect to `model.weights`, updates the weights, and returns the average loss. The order matters. Python constructs the control flow; the tensor framework records differentiable numerical operations. Converting intermediate tensors to Python numbers inside that computation would interrupt the connection needed for gradients.

To connect the classes above, use these source-based definitions in a fresh TensorFlow-backend Keras session. They continue the `NaiveDense` and `NaiveSequential` definitions; the notebook-only `%%backend` wrapper is omitted.

PYTHON

```
import keras
from keras import ops

learning_rate = 1e-3

def update_weights(gradients, weights):
    for g, w in zip(gradients, weights):
        w.assign(w - g * learning_rate)

def one_training_step(model, images_batch, labels_batch):
    with tf.GradientTape() as tape:
        predictions = model(images_batch)
        losses = ops.sparse_categorical_crossentropy(
            labels_batch, predictions
        )
        average_loss = ops.mean(losses)
    gradients = tape.gradient(average_loss, model.weights)
    update_weights(gradients, model.weights)
    return average_loss
```

PYTHON

```
model = NaiveSequential([
    NaiveDense(28 * 28, 512, activation=ops.relu),
    NaiveDense(512, 10, activation=ops.softmax),
])
assert len(model.weights) == 4
```

The model initialization needs `keras` and `ops` available when the constructors execute. The TensorFlow import appears in the tape example above. Alternatively, cell 105 creates `optimizer = keras.optimizers.SGD(learning_rate=1e-3)` and replaces the update body with `optimizer.apply_gradients(zip(gradients, weights))`. Both versions need the gradient-variable pairs in corresponding order. Choose one update function before running the training loop.

In cell 110, the `fit` signature accepts `batch_size`, but the original body constructs `BatchGenerator(images, labels)` without forwarding it. Therefore its parameter has no effect unless it equals the generator's default. The corrected teaching version below preserves the chapter's interface. Dependencies: the original `BatchGenerator` from cell 101, a compatible model, and `one_training_step` from cell 108.

PYTHON

```
def fit(model, images, labels, epochs, batch_size=128):
    for epoch_counter in range(epochs):
        print(f"Epoch {epoch_counter + 1}")
        batches = BatchGenerator(images, labels, batch_size)
        for batch_counter in range(batches.num_batches):
            images_batch, labels_batch = batches.next()
            loss = one_training_step(
                model, images_batch, labels_batch
            )
            if batch_counter % 100 == 0:
                value = float(loss)
                print(f"loss at batch {batch_counter}: {value:.2f}")
```

The new batch generator inside each epoch resets traversal. `%` computes a remainder; `% 100 == 0` logs batches 0, 100, 200, and so on. The displayed epoch adds one for human numbering while the loop remains zero-based. Converting the completed loss to a Python float here is for eager TensorFlow logging after gradient computation; it is

not part of the differentiable calculation. This simple teaching loop does not shuffle examples or provide the features of Keras `Model.fit`.

Notebook locator: "Evaluating the model," cell 113. Predictions have one row per example and one column per class. `argmax(..., axis=1)` removes the class axis by returning the index of each row's greatest value. With no axis, NumPy's `argmax` instead searches the flattened array. Array equality produces Boolean matches; explicitly cast those Booleans before averaging for robust backend behavior.

Corrected teaching adaptation; dependencies: Keras ops and trained model.

```python
predictions = model(test_images)
predicted_labels = ops.argmax(predictions, axis=1)
matches = ops.equal(predicted_labels, test_labels)
accuracy = ops.mean(ops.cast(matches, "float32"))
print(f"accuracy: {float(accuracy):.2f}")
```

True becomes 1 and false becomes 0, so the mean is a fraction correct. An accuracy of `0.96` is a proportion; format it as `{float(accuracy):.1%}` to display `96.0%`. A bare f-string in a notebook displays because the notebook renders its final expression; in a `.py` file you need `print` to show it.

14. Practice before reading the solutions

These exercises use tiny inputs so you can verify every value. Predict first, execute second, then explain any mismatch in one sentence.

1. **Shape detective.** Create `a = np.arange(24).reshape((2, 3, 4))`. Predict `a.ndim`, `len(a)`, `a.size`, and the shapes of `a[1]`, `a[1:2]`, and `a[:, :, -1]`. Which expression preserves the example axis?
2. **Mutation audit.** Create `a = np.arange(6)`, then `b = a[1:4]` and `c = b.copy()`. Assign `b[1] = 90` and `c[0] = -5`. Predict all three arrays. Explain why creating a new variable did not isolate `b`.
3. **Broadcast repair.** For `x = np.arange(12).reshape((3, 4))`, add `[10, 20, 30]` to its rows so the first row gains 10, the second 20, and the third 30. Avoid tiling. State all operand shapes.
4. **Write a numerical function.** Implement `row_sums(x)` with a loop and an accumulator array. Reject non-matrices with `ValueError`. Return one total per row without modifying `x`. Compare against `x.sum(axis=1)` for positive and negative values.
5. **Multiplication and reshape.** With `x = np.array([[1, 2], [3, 4]])` and `v = np.array([10, 20])`, calculate `x * v` and `x @ v` by hand. Then compare `x.reshape((4, 1))` with `x.T.reshape((4, 1))` and explain the changed order.
6. **Callable object.** Write a `Shift` class that stores an amount and returns its input plus that amount when called. Give it a read-only `description` property. Create two objects with different amounts and show their state is independent.
7. **Batch boundary.** Use the `Batches` iterator above with ten examples, matching labels, and size 4. Record batch lengths and label values. Verify a second loop over the same object yields nothing. Explain why a training epoch should create a new instance.
8. **Training-code review.** Explain the fixes for the constant-tensor gradient example and the unused `batch_size` parameter. Then compute accuracy from predictions `[[0.1, 0.9], [0.8, 0.2], [0.3, 0.7]]` with labels `[1, 0, 0]`, using NumPy. State the unrounded accuracy and the two-decimal display.

15. Worked solutions for Part II

1. Shape detective. Rank is 3, first-axis length is 2, and size is 24. The three slice shapes are `(3, 4)`, `(1, 3, 4)`, and `(2, 3)`. `a[1:2]` preserves an example axis of length one; `a[1]` removes it. The final expression removes the last axis because `-1` is one integer index.

2. Mutation audit. `a` becomes `[0, 1, 90, 3, 4, 5]`, `b` becomes `[1, 90, 3]`, and `c` becomes `[-5, 2, 3]`. `c` copied the values before the changes. `b` is a view into `a`, so its element 1 is `a`'s element 2. Separate names and separate storage are different concepts.

3. Broadcast repair. Insert a size-one column axis so `(3, 1)` broadcasts over the four columns of `(3, 4)`.

```python
x = np.arange(12).reshape((3, 4))
offsets = np.array([10, 20, 30])[:, None]
result = x + offsets
assert result.tolist() == [
    [10, 11, 12, 13],
    [24, 25, 26, 27],
    [38, 39, 40, 41],
]
```

`None` inside NumPy indexing inserts a new axis; it is equivalent here to `np.expand_dims(..., axis=1)`. This is a new indexing use of the same Python singleton.

4. A loop with one accumulator per row. The float output is deliberate for this teaching version. Real numerical utilities should choose their dtype according to precision and overflow requirements.

```python
def row_sums(x):
    if x.ndim != 2:
        raise ValueError("Expected a matrix")
    totals = np.zeros(x.shape[0], dtype="float64")
    for i in range(x.shape[0]):
        for j in range(x.shape[1]):
            totals[i] += x[i, j]
    return totals

x = np.array([[1, -2, 3], [-4, 5, 6]])
before = x.copy()
assert np.array_equal(row_sums(x), [2, 7])
assert np.array_equal(row_sums(x), x.sum(axis=1))
assert np.array_equal(x, before)
```

5. Different operations. `x * v` produces `[[10, 40], [30, 80]]`. `x @ v` produces `[50, 110]`. The flattened value orders are `[1, 2, 3, 4]` and `[1, 3, 2, 4]`, respectively. Transpose exchanges rows and columns before reshape reads the values in row order.

6. State plus a callable interface. No numerical framework is necessary here.

```python
class Shift:
    def __init__(self, amount):
        self.amount = amount

    def __call__(self, value):
        return value + self.amount

    @property
    def description(self):
        return f"Add {self.amount}"

small = Shift(2)
large = Shift(10)
assert small(3) == 5
assert large(3) == 13
assert small.description == "Add 2"
```

The property has no setter, so assigning directly to `small.description` raises `AttributeError`. This does not make the whole object immutable: `small.amount` is still an ordinary writable attribute.

7. Exhaustion is state.

```python
iterator = Batches(np.arange(10), np.arange(10), size=4)
seen = [(len(x), y.tolist()) for x, y in iterator]
assert seen == [
    (4, [0, 1, 2, 3]),
    (4, [4, 5, 6, 7]),
    (2, [8, 9]),
]
assert list(iterator) == []
```

This list comprehension evaluates its expression for each pair from the iterator. After the three batches, the index is past the final example, so `__next__` raises `StopIteration`. A new object restores an index of zero.

8. Review and accuracy. Explicitly watch the constant tensor inside the tape, or use a suitable trainable variable. Forward `batch_size` when constructing the batch generator. For the accuracy calculation:

```python
probabilities = np.array([[0.1, 0.9], [0.8, 0.2], [0.3, 0.7]])
labels = np.array([1, 0, 0])
predicted = probabilities.argmax(axis=1)
accuracy = (predicted == labels).astype("float32").mean()
print(predicted.tolist())
print(f"{accuracy:.2f}")
```

Predicted classes are `[1, 0, 1]`; two of three match. The mathematical fraction is `2/3`; the float32 representation is approximately `0.6666667`, and the two-decimal display is `0.67`. Formatting changes the displayed text, not the stored value.

Part III | Python inside chapter 3

Chapter 3 repeats a small learning problem through TensorFlow, PyTorch, JAX, and Keras. This makes it an excellent Python laboratory: the mathematics stays familiar while the way a program stores state, calls functions, and organizes objects changes. Your aim here is to read those decisions confidently.

All cell numbers below are zero-based positions in the local third-edition `chapter03_introduction-to-ml-frameworks.ipynb`, counting both Markdown and code cells. They are not execution counters. Unless marked "notebook excerpt," examples are teaching adaptations. Run each framework section in its own fresh notebook or script, after its imports and setup.

Validation for this part: all 34 Python blocks passed syntax checks. The NumPy exercises, TensorFlow derivatives and 40-step training loop, and custom Keras layer passed local checks. Keras fitting and prediction were checked for one epoch using the TensorFlow backend. PyTorch and JAX examples were reviewed against the notebook and official APIs but were not executed because those packages were unavailable.

16. Recognizing what belongs to Python

Source: "First steps with TensorFlow," cells 11-25; corresponding PyTorch cells 50-61 and JAX cells 84-101.

In `tf.ones(shape=(2, 1))`, Python supplies attribute access, function-call parentheses, a keyword argument, and a tuple. TensorFlow supplies the meaning of `ones` and `shape`. Learning to separate these layers makes an unfamiliar library much less intimidating.

```python
import tensorflow as tf

x = tf.ones(shape=(2, 1))
print(x.shape)
print(x.dtype)
```

`tf` is an ordinary name bound to an imported module. The dot in `tf.ones` retrieves an attribute from that module. Parentheses then call it. In contrast, `x.shape` reads an attribute without a call. `shape` is exposed as a property by many tensor types: its attribute-style interface may compute a result internally. You cannot infer that every attribute is merely a stored value.

The punctuation in `shape=(2, 1)` does not perform a variable assignment in the surrounding scope. Inside a call, it passes a keyword argument named `shape`. The tuple says two rows, one column. `(2, 1)` and `(2,)` describe different ranks; `(2)` is just the integer two in grouping parentheses.

Floating-point notation also matters. `0.`, `0.0`, and `0e0` are float literals. `0` is an integer literal. When differentiation is involved, use floating tensors rather than assuming the framework will differentiate integer data.

Similar mathematics, different function signatures

Intention	TensorFlow	PyTorch	JAX
Ones	`tf.ones(shape=(2, 1))`	`torch.ones(size=(2, 1))`	`jnp.ones(shape=(2, 1))`
Explicit values	`tf.constant([1., 2.])`	`torch.tensor([1., 2.])`	`jnp.array([1., 2.])`
Matrix product	`tf.matmul(a, b)`	`torch.matmul(a, b)`	`jnp.matmul(a, b)`
Join rows	`tf.concat((a, b), axis=0)`	`torch.cat((a, b), dim=0)`	`jnp.concatenate((a, b), axis=0)`

The parameter names are API choices, not interchangeable Python synonyms. Copying `axis=0` into a call that expects `dim=0` may fail. Read the signature with `help(function)` when uncertain. The third column's concatenation example is an extension; the notebook itself does not demonstrate JAX concatenation.

In `torch.rand(3, 1)`, two positional integers specify dimensions. In the notebook's `torch.normal(mean=..., std=...)`, tensors supply elementwise means and standard deviations. Do not generalize the calling convention from one random function to another.

TensorFlow's normal sampler uses `shape`, `mean`, and `stddev`; its uniform sampler uses `shape`, `minval`, and `maxval`. A mean is the distribution's center, while standard deviation describes its spread. A uniform draw uses an interval instead. The spelling `dtype="float32"` passes a string identifier; `dtype=torch.float32` retrieves a dtype object from PyTorch. Those choices are accepted by the relevant APIs, not by a universal Python conversion rule.

The chapter's `square`, `sqrt`, and `+` examples operate element by element. `matmul` combines entries across a shared dimension. Concatenating two `(2, 2)` arrays on axis zero produces `(4, 2)`; it joins existing rows without adding a new axis. Parentheses in `(a, b)` merely package the arguments; the concatenation function decides what to do with them.

17. Building the shared dataset without hidden steps

Source: "An end-to-end example: A linear classifier in pure TensorFlow," cells 34-37. Later framework sections reuse these arrays.

PYTHON

```
import numpy as np

n = 1000
rng = np.random.default_rng(1337)
negative = rng.multivariate_normal(
    mean=[0, 3], cov=[[1, 0.5], [0.5, 1]], size=n
)
positive = rng.multivariate_normal(
    mean=[3, 0], cov=[[1, 0.5], [0.5, 1]], size=n
)
inputs = np.vstack((negative, positive)).astype("float32")
targets = np.vstack((
    np.zeros((n, 1), dtype="float32"),
    np.ones((n, 1), dtype="float32"),
))
assert inputs.shape == (2 * n, 2)
assert targets.shape == (2 * n, 1)
```

This adaptation uses a local random generator so other code does not silently change its random state. A fixed seed makes repeated runs of the same setup reproducible; it does not promise matching random numbers across different libraries.

`cov=[[1, 0.5], [0.5, 1]]` is a list containing two lists. NumPy interprets it as a two-by-two covariance matrix. The two entries in `mean` correspond to the two features, so each generated array has shape `(1000, 2)`.

`np.vstack((negative, positive))` passes one tuple containing two arrays. This is not the same argument structure as passing two separate positional arguments. Stacking produces `(2000, 2)`. `.astype("float32")` is then called on that result. Read chained calls from left to right: build the stacked array, convert its numeric type, bind the result to `inputs`.

Labels have shape `(2000, 1)`, matching the eventual predictions. If you accidentally use `(2000,)`, subtracting it from `(2000, 1)` can broadcast into `(2000, 2000)`. Your code may run while computing the wrong loss. Shape assertions catch a more important problem than a missing semicolon ever would.

18. Reading a plot as a Python expression

Source: cells 37, 43, and 44.

```python
import matplotlib.pyplot as plt

plt.scatter(inputs[:, 0], inputs[:, 1], c=targets[:, 0])
plt.xlabel("Feature 0")
plt.ylabel("Feature 1")
plt.show()
```

`inputs[:, 0]` selects every row and only column zero, giving shape `(2000,)`. The first two arguments are the horizontal and vertical coordinates. `c=` is a keyword understood by Matplotlib, here receiving one value per point for coloring. `plt.show()` asks the plotting system to display the figure; it does not calculate model predictions.

After training, `predictions[:, 0] > 0.5` performs an elementwise comparison. The result is a Boolean array, not one Python Boolean. Passing that array as colors groups points by predicted class. No explicit loop is needed.

In cell 44, `np.linspace(-1, 4, 100)` produces 100 evenly spaced coordinates including the endpoints. The string `"-r"` is a plot-format argument for a red line. It is not a subtraction expression because it is inside quotes.

The boundary equation assumes the second coefficient is nonzero. Reading it as `y = (-w0 / w1) * x + (0.5 - bias) / w1` also reveals operator precedence: multiplication and division bind before addition. A vertical boundary needs separate handling if `w1` is zero. For a PyTorch plot, first use a detached CPU array such as `predictions.detach().cpu().numpy()`; plotting libraries should not receive a tensor still participating in gradient tracking.

19. TensorFlow: names, objects, and mutation

Source: "Tensor assignment and the Variable class," cells 18-22.

```python
import tensorflow as tf

v = tf.Variable(tf.zeros((3, 1)))
alias = v
v.assign(tf.ones((3, 1)))
v[0, 0].assign(3.0)
v.assign_add(tf.ones((3, 1)))
```

`alias = v` does not copy the variable. Both names refer to the same object. `assign` changes that object's stored value, so reading either name sees the update. The final values here are `[[4.], [2.], [2.]]`.

Now compare `v = tf.ones((3, 1))`. This rebinds the name `v` to a tensor. It does not update the previous variable object; `alias` still refers to that object. A Python variable name and a TensorFlow `Variable` are different concepts. Every Python name can be rebound, but a TensorFlow variable additionally supports stored numerical state that assignment methods can change.

The indexed `v[0, 0].assign(...)` form in the example uses TensorFlow's supported variable-slice assignment. Ordinary constant tensors do not support NumPy-style item mutation such as `x[0, 0] = 3`.

This distinction explains training code. `W.assign_sub(change)` modifies the same weight object on each step. `W = W - change` merely binds a local name to a result and can lose the variable behavior the training code expects.

20. `with ... as ...` controls a region of execution

Source: "Gradients in TensorFlow," cells 27-29.

```python
x = tf.Variable(3.0)
with tf.GradientTape() as tape:
    result = tf.square(x)
gradient = tape.gradient(result, x)
```

`with` is Python's context-management statement. It enters a context, runs the indented block, and exits that context even when an exception occurs. `as tape` binds the value returned on entry to the name `tape`. This particular context records eligible tensor operations for differentiation.

Notice where indentation ends. Computing `result` is inside recording; requesting `gradient` is outside. A `with` block does not create a new Python local scope, so both names remain accessible afterward. Moving `result = ...` above the block changes what is recorded even though every name still exists.

A trainable `tf.Variable` accessed inside the tape is watched automatically. A constant needs `tape.watch(x)` before its recorded computation. The default nonpersistent tape normally supports one gradient request. These are TensorFlow rules layered on top of Python's context syntax. TensorFlow GradientTape reference.

Why nested contexts matter

Notebook excerpt, cell 29:

```python
time = tf.Variable(0.0)
with tf.GradientTape() as outer_tape:
    with tf.GradientTape() as inner_tape:
        position = 4.9 * time**2
    speed = inner_tape.gradient(position, time)
acceleration = outer_tape.gradient(speed, time)
```

The inner context finishes before `speed` is computed, but the outer context is still active. That placement lets the outer tape record the derivative calculation. `**` means exponentiation; the first derivative is `9.8 * time`, and its derivative is `9.8`. At `time = 0`, speed is zero without making acceleration zero. Moving the speed calculation outside both contexts would destroy this arrangement.

21. A complete TensorFlow step, read as Python

Source: linear-classifier cells 38-42. Dependencies: TensorFlow and the NumPy `inputs` and `targets` from section 17. The following two blocks belong together.

PYTHON

```
W = tf.Variable(tf.random.uniform((2, 1)))
b = tf.Variable(tf.zeros((1,)))
learning_rate = 0.1

def linear_tf(inputs, W, b):
    return tf.matmul(inputs, W) + b

def mse_tf(targets, predictions):
    return tf.reduce_mean(tf.square(targets - predictions))
```

PYTHON

```
def step_tf(inputs, targets, W, b):
    with tf.GradientTape() as tape:
        predictions = linear_tf(inputs, W, b)
        loss = mse_tf(targets, predictions)
    grad_W, grad_b = tape.gradient(loss, [W, b])
    W.assign_sub(learning_rate * grad_W)
    b.assign_sub(learning_rate * grad_b)
    return loss

for step in range(40):
    loss = step_tf(inputs, targets, W, b)
    print(f"Step {step}: {float(loss):.4f}")
```

`W` and `b` are TensorFlow variables; `predictions` is a tensor of shape `(2000, 1)`; `loss` is a scalar tensor with shape `()`. TensorFlow converts these NumPy inputs when its operations consume them. The weights have shape `(2, 1)` and the bias has shape `(1,)`, so the bias broadcasts across all rows.

`[W, b]` groups the two differentiation targets in order. The return value contains corresponding gradients. `grad_W, grad_b = ...` unpacks it; Python requires exactly two elements. An unconnected gradient can be `None`, so a later multiplication error may actually indicate a broken dependency earlier in the computation.

The learning rate is read from surrounding module scope. The function does not assign to that name, so it needs no `global` statement. Parameters `W` and `b` are local names referring to the caller's variable objects. Mutation explains why the caller sees updated weights even though only `loss` is returned.

The notebook's cell 41 passes `predictions, targets` to a function declared as `mean_squared_error(targets, predictions)`. Squaring makes this particular loss symmetric, so the number is unchanged. Preserve the intended argument order anyway; other losses are not symmetric.

22. Decorators: passing a function instead of calling it

Source: TensorFlow cells 31-32, PyTorch 78-79, JAX 112 and 117.

PYTHON

```
@tf.function
def twice(x):
    return 2 * x
```

Conceptually, Python creates the function and then performs `twice = tf.function(twice)`. The decorator receives the function object. `twice(3)` would instead call the function and produce a value. Omitting parentheses can be exactly the right thing when an API expects behavior to wrap or transform.

For `@tf.function(jit_compile=True)`, Python first calls `tf.function` with the configuration. The resulting decorator receives the function. At the language level, a decorated definition is convenient spelling for function transformation and rebinding.

The `@` in `a @ b` means matrix multiplication. Its position between expressions distinguishes it from a decorator line above `def`. The character is shared; the operations are different.

`torch.compile(model)` similarly receives an existing callable object and returns a compiled callable. A model instance can be callable without being a plain function. `@jax.jit` decorates a function using the same Python syntax.

Compilation is optional for learning these steps. TensorFlow and JAX may trace Python functions and reuse compiled computations, so Python side effects such as `print` or mutation of an ordinary list are not reliable per-step logging mechanisms inside traced code. Keep simple reporting outside and establish a working uncompiled version first. Compilation support also depends on the operations and environment. TensorFlow function guide, JAX JIT guide.

23. PyTorch: attributes that store gradients

Source: "Computing gradients with PyTorch," cells 63-65.

PYTHON

```
import torch

x = torch.tensor(3.0, requires_grad=True)
torch.square(x).backward()
print(x.grad)                    # tensor(6.)
torch.square(x).backward()
print(x.grad)                    # tensor(12.)
x.grad = None
```

`requires_grad=True` configures this tensor for gradient tracking. `backward()` is a method call on the result tensor. It has a side effect: relevant leaf tensors accumulate gradient values in `.grad`. The method does not return the gradient for you to assign.

The second square is a new forward computation. Its gradient adds to the previous one, producing twelve. This differs from trying to backpropagate through the same already-freed graph twice. Explicit accumulation can be useful, but this chapter's training loop wants fresh gradients each step. PyTorch backward reference.

`None` is Python's singleton for absence, not numeric zero. `x.grad = None` assigns an attribute, clearing the stored gradient. `x = None` would instead discard the local binding to the tensor. `x.grad == None` would be a comparison, not a reset. Use `x.grad is None` when checking absence.

The notebook's dense helpers in cells 61 and 79 use `torch.nn.relu`, which is not the functional API. Use `torch.nn.functional.relu(...)` or `torch.relu(...)`; the module class is `torch.nn.ReLU`. Capitalization and namespaces are part of an API's contract. PyTorch functional ReLU.

24. A corrected PyTorch step with explicit state

Source: "An end-to-end example: A linear classifier in pure PyTorch," cells 67-70. Dependencies: section 17 data and `import torch`. The function name is deliberately distinct from later model objects.

PYTHON

```
torch_inputs = torch.tensor(inputs, dtype=torch.float32)
torch_targets = torch.tensor(targets, dtype=torch.float32)
W = torch.rand(2, 1, requires_grad=True)
b = torch.zeros(1, requires_grad=True)
learning_rate = 0.1

def linear_torch(inputs, W, b):
    return inputs @ W + b

def mse_torch(targets, predictions):
    return torch.mean(torch.square(targets - predictions))
```

PYTHON

```
def step_torch(inputs, targets, W, b):
    predictions = linear_torch(inputs, W, b)
    loss = mse_torch(targets, predictions)
    loss.backward()
    with torch.no_grad():
        W -= learning_rate * W.grad
        b -= learning_rate * b.grad
    W.grad = None
    b.grad = None
    return loss

for step in range(40):
    loss = step_torch(torch_inputs, torch_targets, W, b)
    print(f"Step {step}: {loss.item():.4f}")
```

Cell 70 calls `model(inputs)` even though cell 68 defines `model(inputs, W, b)`. In that sequence Python raises `TypeError` for missing arguments. The adaptation supplies all three arguments and converts the shared NumPy arrays explicitly to PyTorch tensors.

For these tensors, `-=` performs an in-place update. It preserves the weight objects used by the caller. This statement is inside `torch.no_grad()` so the parameter update itself is not added to the reverse-mode gradient computation. The context has no `as` clause because we do not need a context-bound name. PyTorch no_grad reference.

After leaving the context, gradient tracking resumes for ordinary operations. Clearing gradients belongs after their use or before the next backward pass, never between `backward()` and the update. `.item()` extracts a Python scalar from a one-element tensor for formatting. Do not replace the tensor loss by that scalar before `backward()`; Python floats have no gradient history.

25. Inheritance packages state with behavior

Source: "Packaging state and computation with the Module class," cells 72-76.

```python
class LinearModel(torch.nn.Module):
    def __init__(self, input_dim=2, output_dim=1):
        super().__init__()
        self.W = torch.nn.Parameter(
            torch.rand(input_dim, output_dim)
        )
        self.b = torch.nn.Parameter(torch.zeros(output_dim))

    def forward(self, inputs):
        return inputs @ self.W + self.b

torch_model = LinearModel()
optimizer = torch.optim.SGD(torch_model.parameters(), lr=0.1)
```

The parentheses in `class LinearModel(torch.nn.Module)` declare inheritance. `super().__init__()` initializes the inherited module machinery before registered parameters are assigned. `self` is a conventional parameter name for the current instance; Python supplies that instance when you call a bound method.

`self.W` persists as an attribute between calls. A bare local `W` inside `__init__` would disappear as a local binding when initialization finishes unless something retained it. Assigning a `Parameter` to a module attribute allows the module to discover it. `parameters()` is a method returning an iterator over registered parameters; the optimizer consumes those objects. It does not receive the method object because the parentheses are present.

Call `torch_model(torch_inputs)`, rather than directly calling `.forward(torch_inputs)`. Here `torch_inputs` is the PyTorch tensor converted in the preceding section; the shared `inputs` is still a NumPy array. The inherited `__call__` behavior routes through module machinery and then invokes your `forward`. Python's callable-object protocol explains the function-like spelling. Keras uses the same outward spelling but a different customization method named `call`.

```python
def step_module(inputs, targets):
    optimizer.zero_grad(set_to_none=True)
    predictions = torch_model(inputs)
    loss = mse_torch(targets, predictions)
    loss.backward()
    optimizer.step()
    return loss
```

This version reads the model and optimizer from surrounding scope. It also uses `mse_torch` from the preceding section. Call it with `loss = step_module(torch_inputs, torch_targets)`. Moving the model and optimizer into arguments would make dependencies more explicit. The notebook resets with `model.zero_grad()` afterward; either placement can work when gradients are cleared between steps. Do not apply the manual update as well as `optimizer.step()` unless you intentionally want two updates.

26. JAX: familiar syntax, immutable arrays

Source: "Tensors in JAX" and "Tensor assignment," cells 84-86 and 98-100.

```python
import jax
from jax import numpy as jnp

x = jnp.array([1, 2, 3], dtype="float32")
old_x = x
new_x = x.at[0].set(10)
x = x * 2
```

`from jax import numpy as jnp` imports JAX's NumPy-like namespace and gives it a short local name. It does not turn ordinary NumPy into JAX. The functions look familiar, but their returned objects have different behavior.

`x.at[0].set(10)` returns an updated array, leaving the original array unchanged. Read the chain as attribute access, indexing of the update helper, then a method call. Afterward, `old_x` still denotes `[1, 2, 3]`, `new_x` denotes `[10, 2, 3]`, and rebound `x` denotes `[2, 4, 6]`. JAX indexed-update reference.

Cell 100 uses `e *= d`. For immutable JAX arrays, this produces a result and rebinds `e`; it does not mutate the array previously referred to by `e`. Never infer mutability solely from `+=` or `*=`. Python lets an object's type define augmented-assignment behavior.

This is why the JAX training step must return updated weights and why the caller must capture them. TensorFlow variables and PyTorch parameters in the earlier examples support object mutation; JAX's array-based example communicates state through returned values.

27. Random state can be implicit or explicit

Source: "Random number generation in JAX," cells 88-96.

The notebook first demonstrates NumPy's global random generator. Two calls consume successive state, while resetting the seed makes a sequence repeat. Its cell 90 contains an argument-binding mistake: `np.random.normal((3,))` passes `(3,)` as `loc`, the distribution's mean. The requested three-element output needs `size=(3,)`. NumPy normal signature.

An adaptation that states both the input shape and random dependency is:

```python
def apply_noise(x, seed):
    rng = np.random.default_rng(seed)
    return x * rng.normal(size=x.shape)

x = np.ones(3, dtype="float32")
y = apply_noise(x, 1337)
z = apply_noise(x, 1338)
```

Here `seed` is a parameter, and `rng` is a local object. The function leaves `x` unchanged because it returns multiplication's result. Passing the same seed and input repeats the result. This uses multiplication by noise, not addition of noise.

JAX makes a random key an explicit input:

```python
key = jax.random.key(1337)
key, draw_key = jax.random.split(key)
noise = jax.random.normal(draw_key, shape=(3,))
key, next_draw_key = jax.random.split(key)
next_noise = jax.random.normal(next_draw_key, shape=(3,))
```

Each split yields new keys. Unpacking keeps one for future splitting and uses another for the draw. The right-hand side uses the old value of `key` before the left-hand assignments happen. Reusing an identical key with an identical sampling call repeats its result; drawing does not automatically advance a hidden generator. JAX pseudorandom-number guide.

Cell 96 uses `split(..., num=1)[0]`: get a collection containing one new key, then retrieve its first element. The two-key pattern above makes ongoing state management clearer. A seed integer and a JAX key object are not interchangeable merely because both relate to randomness.

28. Functions can accept and return functions

Source: "Computing gradients with JAX," cells 103-108, and classifier cells 114-117.

```python
def square_loss(x):
    return jnp.square(x)

grad_fn = jax.grad(square_loss)
gradient = grad_fn(jnp.array(3.0))
value_and_grad_fn = jax.value_and_grad(square_loss)
value, gradient = value_and_grad_fn(jnp.array(3.0))
```

At the first assignment, `square_loss` is passed without parentheses. JAX receives the function itself and creates another function. The next line calls that derivative function on a value. The results at three are value nine and gradient six. Function names are ordinary bindings: they can be passed, assigned, and replaced just like other objects.

For `value_and_grad`, the differentiated function must return a scalar. An array of shape `(1,)` is still a vector, even if it has one element. `jnp.mean(...)` in the classifier creates the required scalar. By default the derivative is with respect to the first positional argument; `argnums` selects others. A tuple of weight arrays can occupy that single argument. JAX value_and_grad reference.

Tuple state and nested return values

```python
def loss_jax(state, inputs, targets):
    W, b = state
    predictions = inputs @ W + b
    return jnp.mean(jnp.square(targets - predictions))

loss_and_grads_jax = jax.value_and_grad(loss_jax)
```

`state` is a tuple holding two arrays. Unpacking it makes convenient local names without copying either array. The resulting gradients mirror that tuple, so calling `loss_and_grads_jax((W, b), inputs, targets)` returns `(loss, (grad_W, grad_b))`. Its distinct name avoids overwriting the earlier scalar derivative function when notebook cells are rerun.

For a diagnostic extension, return `loss, predictions` and construct the transformation with `has_aux=True`. Then unpack `(loss, predictions), grads = transformed(...)`. This flag tells JAX which output is the scalar objective; it does not ask JAX to differentiate the predictions as a second loss.

29. A complete JAX step returns its new state

Source: classifier cells 117-118. Dependencies: section 17 data, JAX imports, and `loss_jax`/`loss_and_grads_jax` from section 28.

PYTHON

```
jax_inputs = jnp.asarray(inputs)
jax_targets = jnp.asarray(targets)
key = jax.random.key(1337)
W = jax.random.uniform(key, shape=(2, 1))
b = jnp.zeros((1,), dtype="float32")
learning_rate = 0.1

@jax.jit
def step_jax(inputs, targets, W, b):
    loss, grads = loss_and_grads_jax((W, b), inputs, targets)
    grad_W, grad_b = grads
    W = W - learning_rate * grad_W
    b = b - learning_rate * grad_b
    return loss, W, b

for step in range(40):
    loss, W, b = step_jax(jax_inputs, jax_targets, W, b)
    print(f"Step {step}: {float(loss):.4f}")
```

Every update binds local names to new array values. `return loss, W, b` returns a three-element tuple even without explicit outer parentheses. The assignment in the loop unpacks that tuple, updating the caller's bindings. If you instead write only `loss = step_jax(...)`, `loss` becomes the entire tuple and the caller's `W` and `b` remain unchanged.

The notebook creates `state = (W, b)` before its loop but then passes `W` and `b` directly. That old tuple does not automatically refresh when the names are rebound. Either reconstruct a tuple when needed, as the function does, or make the training API accept and return `state` consistently.

The setup here samples JAX weights directly and explicitly converts inputs. Cell 118 instead builds arrays from NumPy draws; those values are valid inputs, but JAX dtype defaults can affect conversion. Explicit floating dtypes make teaching examples easier to inspect.

30. Keras: a callable class is a learning building block

Source: "Picking a backend framework," cell 123; "The base Layer class," cells 126-127.

In a fresh process, set `os.environ["KERAS_BACKEND"]` before importing Keras. That dictionary-like assignment configures an environment variable; it is not a general Python switch that converts existing tensors. This course's Keras examples can use the notebook's JAX backend.

In this fresh process, rerun the shared dataset block in section 17 as well. It defines `inputs`, `targets`, and the NumPy generator `rng` used in the validation-split example. Each other fresh framework environment likewise needs that data setup before its classifier examples.

PYTHON

```
import os
os.environ["KERAS_BACKEND"] = "jax"
import keras
```

The following corrected adaptation is a complete class:

```python
class SimpleDense(keras.Layer):
    def __init__(self, units, activation=None):
        super().__init__()
        self.units = units
        self.activation = keras.activations.get(activation)
    def build(self, input_shape):
        input_dim = input_shape[-1]
        self.W = self.add_weight(
            shape=(input_dim, self.units),
            initializer="random_normal",
        )
        self.b = self.add_weight(
            shape=(self.units,), initializer="zeros"
        )
    def call(self, inputs):
        y = keras.ops.matmul(inputs, self.W) + self.b
        return self.activation(y)
```

`__init__` stores configuration. `build` creates shape-dependent weights. `call` describes the numerical computation. The inherited `__call__` machinery coordinates building on first use and then invokes `call`; use `layer(inputs)` to enter that machinery. `add_weight` creates weights registered with the layer, while ordinary local arrays would not provide that same registration. Keras subclassing guide.

The notebook unpacks `batch_dim, input_dim = input_shape`, assuming a rank-two input. `input_shape[-1]` selects the final dimension directly and avoids an unused batch variable. For input `(2, 784)` and 32 units, `W` is `(784, 32)`, `b` is `(32,)`, and the output is `(2, 32)`.

An activation string is not a function

Cell 126 stores `activation` directly, then calls it. Cell 127 passes the callable `keras.ops.relu`, so this works. Cell 131 passes `"relu"` and `"softmax"` strings to that custom class. A string cannot be called; using the layer will fail.

The corrected class resolves the identifier with `keras.activations.get`. This accepts supported string identifiers or callables; `None` resolves to the identity activation. That is why the adapted `call` no longer needs the notebook's `if self.activation is not None:` guard. Keras activation resolver source.

```python
layer = SimpleDense(32, activation="relu")
x = keras.ops.ones((2, 784))
y = layer(x)
print(y.shape)                          # (2, 32)
```

The expression `keras.ops.relu` retrieves a function; `keras.ops.relu(x)` calls it. The expression `"relu"` creates text. A library can choose to translate that text into a callable, but Python does not do so automatically.

31. Lists of objects and training configuration

Source: "Automatic shape inference," cells 129-131, and "The compile step," cells 134-135.

```python
keras_model = keras.Sequential([
    SimpleDense(32, activation="relu"),
    SimpleDense(10, activation="softmax"),
])
```

Square brackets build a list of two already-created layer objects. Parentheses outside the brackets pass that list to the `Sequential` constructor. Its ordering determines computation order. The second layer infers its input width from the first layer's output.

For the shared two-feature, binary-label dataset, use the notebook's one-output model instead:

```python
keras_model = keras.Sequential([keras.layers.Dense(1)])
keras_model.compile(
    optimizer=keras.optimizers.RMSprop(learning_rate=0.1),
    loss=keras.losses.MeanSquaredError(),
    metrics=[keras.metrics.BinaryAccuracy()],
)
```

Each constructor call creates an object. `metrics=[...]` supplies a list because several metrics may be requested. The string alternatives in cell 134 are identifiers that Keras resolves, just as built-in activation arguments are resolved. Passing an instantiated optimizer gives you a place to configure `learning_rate` explicitly.

This particular `compile` method configures training: optimizer, loss, metrics, and execution settings. It is not the same API as `torch.compile`. Training starts with `fit`. A linear output with MSE reproduces the chapter's teaching example; it is not a guarantee that its outputs are probabilities. Keras model training APIs.

32. Shuffle paired data with one permutation

Source: "Monitoring loss and metrics on validation data," cell 141.

```python
permutation = rng.permutation(len(inputs))
shuffled_inputs = inputs[permutation]
shuffled_targets = targets[permutation]

n_val = int(0.3 * len(inputs))
val_inputs = shuffled_inputs[:n_val]
val_targets = shuffled_targets[:n_val]
train_inputs = shuffled_inputs[n_val:]
train_targets = shuffled_targets[n_val:]
```

`len(inputs)` returns its first-axis length, here 2000. The permutation is an integer array containing every row index once. Indexing with that array performs advanced indexing: select rows in the specified order, creating new NumPy arrays. Reusing the same order for targets preserves each example-label pair. Independent shuffles silently damage the dataset.

`0.3 * len(inputs)` is a floating-point calculation. `int(...)` converts it to an integer by truncating toward zero. With a nonnegative count this discards any fractional remainder. Python slice endpoints need integer-like values, not `600.0` merely because it represents a whole number.

The slice `[:n_val]` excludes index `n_val`; `[n_val:]` starts there. This partitions the shuffled arrays without overlap. Basic NumPy slices generally produce views, so avoid modifying these subsets in place if you need the backing shuffled arrays preserved. Validation has 600 samples and training has 1400 here.

Shuffling before splitting matters because the original dataset puts all zero labels before all one labels. This random split is appropriate for this synthetic example; it is not a universal instruction to shuffle time-dependent datasets.

33. Return objects, dictionaries, and prediction batches

Source: "Understanding the fit method," cells 138-139, and inference cell 143.

```python
history = keras_model.fit(
    train_inputs, train_targets,
    epochs=5, batch_size=16,
    validation_data=(val_inputs, val_targets),
)
losses = history.history["loss"]
last_loss = losses[-1]
predictions = keras_model.predict(val_inputs, batch_size=128)
print(predictions[:10])
```

`validation_data=(val_inputs, val_targets)` is one keyword argument whose value is a pair. It does not pass two independent validation arguments. `epochs=5` repeats training over the dataset five times; `batch_size=16` groups training examples for each step. Prediction batching controls how examples are processed, not how many predictions are ultimately returned.

The left `history` is your variable name. The right `.history` is an attribute on the returned History object. It holds a dictionary mapping names to lists of epoch results. `history.history["loss"]` first retrieves the dictionary, then looks up a string key. `[-1]` retrieves the final list element. A missing dictionary key raises `KeyError`; it does not return an empty list.

Inspect actual metric keys instead of guessing:

```python
for name, values in history.history.items():
    print(f"{name}: final value = {values[-1]:.4f}")
```

`.items()` returns key-value pairs that the loop unpacks into two names. This pattern also works for ordinary dictionaries unrelated to machine learning. The result of `predict` for the one-output model has shape `(600, 1)`; `predictions[:10]` keeps the first ten rows and the column dimension.

34. Exercises: make the state changes explicit

Attempt these before reading section 35. Explain each answer in words as well as code.

Exercise A - Repair three calls

Explain and repair `np.random.normal((3,))`, `torch.nn.relu(x)`, and `linear_torch(inputs)` when the function definition requires `inputs, W, b`. For the last call, also identify the tensor-type requirement if `inputs` came from NumPy.

Exercise B - Follow aliases

For TensorFlow, create `v = tf.Variable(1.0)` and `old = v`, then call `v.assign_add(2.0)` and finally assign `v = tf.constant(9.0)`. Predict the value and object kind referred to by each name. Repeat the reasoning for JAX with `x = jnp.array([1.0])`, `old = x`, and `x += 2`.

Exercise C - Catch an accidental outer difference

You have predictions with shape `(4, 1)` and targets with shape `(4,)`. Predict the shape of `targets - predictions`. Write a small NumPy example that demonstrates it, then repair the targets without changing which sample each value belongs to.

Exercise D - Debug gradient accumulation

For `x = torch.tensor(2.0, requires_grad=True)`, run a newly computed `x**2` backward three times without clearing gradients. Predict `.grad` after each run. Then write a loop in which each iteration prints only that iteration's gradient.

Exercise E - Refactor a JAX step

Rewrite `step_jax` to accept `(state, inputs, targets)` and return `(loss, new_state)`. Show the calling loop. Explain why creating `state = (W, b)` once does not keep it synchronized with future rebindings of `W` and `b`.

Exercise F - Verify paired shuffling

Make five input rows containing IDs zero through four. Define each target as ten times its row's ID. Shuffle both arrays using one permutation, split off two validation rows, and assert the target relationship still holds in both subsets.

Exercise G - Make an activation contract visible

Create the corrected `SimpleDense` with `activation=None`, `activation="relu"`, and a callable activation. Explain when each activation is resolved, when weights are created, and which method actually implements the tensor arithmetic. Predict the weight and output shapes for four input rows, three features, and five units.

Exercise H - Read a training history

Given `logs = {"loss": [0.8, 0.4, 0.3], "val_loss": [0.9, 0.5, 0.6]}`, print the epoch with the lowest validation loss using one-based numbering. Explain why the last training loss and the lowest validation loss need not occur at the same epoch.

35. Worked solutions and reasoning

Solution A - Argument binding precedes intention

Use `np.random.normal(size=(3,))`, `torch.nn.functional.relu(x)`, and `linear_torch(torch_inputs, W, b)`. Build `torch_inputs = torch.tensor(inputs, dtype=torch.float32)` if needed. Python binds positional arguments by position; it cannot infer that `(3,)` was intended as a size. Attribute lookup must find the named function before a call can occur. A function with three required parameters needs all three supplied.

Solution B - Names are not storage containers

After TensorFlow mutation, `old` and `v` refer to the same variable containing `3.0`. After rebinding, `v` refers to a constant tensor containing `9.0`, while `old` still refers to the variable containing `3.0`. In JAX, `old` remains `[1.0]`; `x` is rebound to an array containing `[3.0]`. The augmented assignment cannot mutate an immutable array.

Solution C - Shape alignment can conceal a bug

```python
p = np.array([[10.], [20.], [30.], [40.]])
t = np.array([1., 2., 3., 4.])
assert (t - p).shape == (4, 4)
t = t.reshape(-1, 1)
assert (t - p).shape == (4, 1)
```

The original target is treated as a row during broadcasting, so every prediction is compared with every target. `reshape(-1, 1)` lets NumPy infer the row count and preserves element order. A working subtraction is not proof of a meaningful subtraction.

Solution D - Clear before the next accumulation

The successive gradients are four, eight, and twelve. Each fresh computation contributes another `2*x = 4`.

```python
x = torch.tensor(2.0, requires_grad=True)
for _ in range(3):
    x.grad = None
    loss = x**2
    loss.backward()
    print(x.grad.item())              # 4.0 every time
```

The underscore is an ordinary variable name conventionally indicating an unused loop value. No parameter update occurs in this exercise, so the derivative's input stays two.

Solution E - Return the container you want to preserve

```python
@jax.jit
def step_state(state, inputs, targets):
    loss, grads = loss_and_grads_jax(state, inputs, targets)
    W, b = state
    grad_W, grad_b = grads
    new_state = (
        W - learning_rate * grad_W,
        b - learning_rate * grad_b,
    )
    return loss, new_state

state = (W, b)
for _ in range(40):
    loss, state = step_state(state, jax_inputs, jax_targets)
```

A tuple holds references to the objects supplied when it is constructed. It does not hold live references to the future meanings of variable names. Capturing `new_state` makes state propagation explicit and prevents accidentally training from the original weights repeatedly.

Solution F - One index order protects the relationship

```python
rows = np.arange(5).reshape(-1, 1)
labels = 10 * rows
order = np.random.default_rng(7).permutation(len(rows))
rows, labels = rows[order], labels[order]
val_x, val_y = rows[:2], labels[:2]
train_x, train_y = rows[2:], labels[2:]
assert np.array_equal(val_y, 10 * val_x)
assert np.array_equal(train_y, 10 * train_x)
```

The relationship test is stronger than merely checking equal lengths. Independent permutations would usually preserve length while corrupting pairings. Both right-hand indexing expressions are evaluated before either left-hand binding changes.

Solution G - Initialization, building, and calling differ

The constructor resolves the activation using `keras.activations.get`. `None` becomes an identity callable; the string identifies a supported activation; a supplied callable already represents behavior. First calling the layer with inputs triggers weight building. The layer's `call` method performs matrix multiplication, bias addition, and activation. For `(4, 3)` inputs and five units, weights are `(3, 5)`, bias is `(5,)`, and output is `(4, 5)`.

```python
for activation in (None, "relu", keras.ops.relu):
    layer = SimpleDense(5, activation=activation)
    result = layer(keras.ops.ones((4, 3)))
    assert tuple(result.shape) == (4, 5)
```

Solution H - The best item can be located by its value

```python
logs = {
    "loss": [0.8, 0.4, 0.3],
    "val_loss": [0.9, 0.5, 0.6],
}
val_losses = logs["val_loss"]
best_index = min(range(len(val_losses)), key=val_losses.__getitem__)
print(best_index + 1)                # 2
```

`range` supplies candidate indices. `key=` receives a callable that tells `min` how to score each index. `val_losses.__getitem__` is the list's bound indexing method; passing it does not call it immediately. An easier equivalent is `best_index = val_losses.index(min(val_losses))`, which chooses the first occurrence if the minimum is tied. Training loss and validation loss measure different data, so one can decrease while the other rises.

Part IV | Read, repair, and rebuild

36. Decode the notebook's backend helper

Source: cell 3 in both notebooks. This helper is easy to skip, but it contains several useful Python patterns. The version below preserves its behavior and splits a long assignment for readability. It runs inside IPython/Jupyter; `get_ipython()` is supplied by that environment.

```python
import os
from IPython.core.magic import register_cell_magic

@register_cell_magic
def backend(line, cell):
    current = os.environ.get("KERAS_BACKEND", "")
    required = line.split()[-1]
    if current == required:
        get_ipython().run_cell(cell)
    else:
        print(
            f"This cell requires the {required} backend. "
            f"Set KERAS_BACKEND to \"{required}\", "
            "restart the runtime, and rerun the notebook."
        )
```

`def` creates a function object; it does not immediately execute the function body. `@register_cell_magic` applies a decorator that registers the function with IPython. When a user runs a cell beginning with `%%backend tensorflow`, IPython supplies the text after the magic name as `line`, and the remainder of the cell as the string `cell`. The decorator is an API provided by IPython; the decorator syntax itself belongs to Python.

`os.environ.get("KERAS_BACKEND", "")` looks up a key and returns an empty string if that key is missing. Unlike `os.environ["KERAS_BACKEND"]`, it avoids a `KeyError` for an absent setting. The empty string is a default value, not an exception handler.

`line.split()` splits a string on whitespace and returns a list of words. `[-1]` selects its last word. For `line = "tensorflow"`, the list is `["tensorflow"]`. If the string has no words, indexing the empty list raises `IndexError`. The helper assumes you supply a backend name.

`if current == required:` compares two strings. The indented body executes only if they compare equal. `else:` handles the other case. `get_ipython()` returns an object; `.run_cell(cell)` calls a method on that object. This evaluates the cell's text in the interactive environment. It does not change Keras's backend by itself.

The three adjacent strings inside `print(...)` form a single string. Python joins adjacent string literals, even across physical lines inside parentheses. Each `f` prefix applies only to its own literal. `\"` embeds a literal double quote inside a double-quoted string. An alternative would be a single-quoted outer string containing double quotes.

The source writes `current, required = expression1, expression2` on one line. That is the same unpacking pattern you saw with MNIST. Our adaptation uses two assignments to make evaluation easier to follow. The comment `# @title` in the source does not decorate the function; the non-comment line starting with `@register_cell_magic` does.

Micro-exercise. Predict `" use tensorflow ".split()[-1]`. Then explain why `" ".split()[-1]` fails. Answer: the first produces `"tensorflow"`; the second tries to read the last item of an empty list.

37. A debugging routine you can reuse

When a cell fails, read the final exception message first, then trace back to the line in your code that caused it. Do not respond by rerunning random cells. Check what values the failing expression actually received.

```python
print(type(inputs))
print(inputs.shape)
print(inputs.dtype)
print(type(model))
```

These checks distinguish a NumPy array from a framework tensor, a two-dimensional matrix from a one-dimensional vector, and a function from a model instance. Add them immediately before the failing line. Remove or reduce them when you understand the issue.

Symptom	Likely Python issue	First useful check
`NameError`	A name was never bound in this kernel	Run its definition or import cell
Missing positional arguments	The call does not match the definition	Compare the call with `def`
`str` object is not callable	Code uses a string as if it were a function	Inspect the activation's value and type
`AttributeError`	Object has no requested attribute	Check the module and spelling
Cannot reshape array	Element counts do not match	Compare old size with new shape product
Broadcasting error	Trailing dimensions do not align	Write both shapes side by side
Ambiguous array truth value	`if` received many Booleans	Choose an explicit `.any()` or `.all()`
Gradient is `None`	The framework did not track a path	Check variable status, watch, and graph
Loss changes unexpectedly on rerun	The kernel retained model or gradient state	Recreate the intended starting state

`help(function)` displays documentation for the function object. `type(object)` identifies its type. `dir(object)` lists available names and is useful for exploration, though it can be long. Read the signature before changing a call: a positional tuple can accidentally be bound to the wrong parameter, as in the chapter 3 noise example.

A shape mismatch that does not raise an error

```python
import numpy as np
targets = np.array([0.0, 1.0, 0.0])
predictions = np.array([[0.1], [0.9], [0.2]])
wrong = targets - predictions
right = targets.reshape((-1, 1)) - predictions
print(wrong.shape, right.shape)
```

This prints `(3, 3) (3, 1)`. Broadcasting can create a valid array with the wrong meaning. For a per-example loss, you want each prediction paired with its matching target. Shape checks belong to understanding the program, not just fixing exceptions. `-1` in a reshape asks NumPy to infer that dimension from the number of elements.

Assertions versus validation

The source uses `assert` to express assumptions while teaching, such as requiring a matrix. An assertion that is false raises `AssertionError`. Python can disable assertions under optimized execution, so use explicit validation for a reusable public function.

```python
def require_positive_batch_size(batch_size):
    if batch_size <= 0:
        raise ValueError("batch_size must be positive")
    return batch_size
```

This is a small extension beyond the notebook. `raise` deliberately reports an invalid value. The message explains what the caller must fix. An exception interrupts the normal path; `return` is reached only for acceptable input.

38. Repair cards for the supplied notebooks

These observations apply to the local third-edition files inspected for this course. The original notebooks were left unchanged. Each repair is explained in the relevant lesson as well.

Source	What needs attention	Repair
Ch. 2 cell 107	A constant is not watched by the tape	Call `tape.watch(x)` inside the context
Ch. 2 cell 110	`fit` accepts `batch_size` but does not forward it	Pass it into `BatchGenerator`
Ch. 3 cells 61, 79	`torch.nn.relu` is not the functional API	Use `torch.nn.functional.relu`
Ch. 3 cell 70	Functional `model` needs three arguments	Call `model(inputs, W, b)`
Ch. 3 cell 90	`(3,)` is passed as the distribution mean	Use `size=(3,)` or `size=x.shape`
Ch. 3 cells 126, 131	Custom layer calls activation strings	Resolve with `keras.activations.get`

Two further habits improve clarity. The TensorFlow training step calls mean squared error with predictions and targets reversed; squaring the difference makes that particular result the same, but consistent argument order prevents bugs with asymmetric losses. The chapter 2 accuracy example reduces Boolean matches; explicitly casting matches to a floating dtype makes the intended numeric mean clear across backends.

Some examples are intentionally minimal. `BatchGenerator.next()` is not Python's iterator protocol, even though the class is called a generator. A raw `model` score from the linear classifier is not automatically a probability. A timing comparison does not promise a fixed speedup on every computer. Separating Python behavior from library behavior helps you read such simplifications accurately.

39. Final lab: build a tiny prediction pipeline

This lab combines the Python patterns from both chapters without downloading data or installing a deep-learning framework. It requires only NumPy. Run the following blocks in order in a fresh notebook. The deliberately chosen weights are fixed; this is a programming exercise, not a trained classifier.

Your task before reading the solution

1. Create five tiny grayscale images and five labels. Convert the images to floating point, flatten each image, and scale pixel values into the interval from 0 to 1.
2. Build a callable class that stores a weight matrix and bias, computes `inputs @ W + b`, and exposes a read-only `weights` property returning a list.
3. Write a batch function that returns image and label slices. Batch size 2 should produce lengths 2, 2, and 1.
4. Compute predicted classes using `argmax` along the class axis. Compare with labels and return both accuracy and predictions.
5. Test that preprocessing does not change the original pixel array. Explain why a tied maximum chooses the first class here.

Expected checkpoints: features have shape `(5, 4)` and dtype `float32`; scores have shape `(5, 2)`; predicted labels are `[1, 0, 1, 0, 0]`; accuracy is 0.8. Use these checkpoints to locate a mistake, rather than replacing your entire solution.

Worked solution, step 1: data and preprocessing

PYTHON

```
import numpy as np

images = np.array([
    [[0, 255], [0, 255]],
    [[255, 0], [255, 0]],
    [[0, 128], [0, 128]],
    [[128, 0], [128, 0]],
    [[0, 0], [0, 0]],
], dtype="uint8")
labels = np.array([1, 0, 1, 0, 1])
original = images.copy()

def prepare(images):
    flat = images.reshape((len(images), -1))
    return flat.astype("float32") / 255

features = prepare(images)
assert features.shape == (5, 4)
assert features.dtype == np.float32
assert np.array_equal(images, original)
```

Each inner two-by-two list is one image. `len(images)` counts images, not pixels. Flattening preserves the leading sample axis. The dtype conversion creates a separate floating-point array, so the returned normalized values do not overwrite the original integer pixels. `np.array_equal` is a single Boolean comparison of whole-array equality, suitable for an assertion.

Worked solution, step 2: a callable object

PYTHON

```
class TinyDense:
    def __init__(self, W, b):
        self.W = W.copy()
        self.b = b.copy()

    def __call__(self, inputs):
        return inputs @ self.W + self.b

    @property
    def weights(self):
        return [self.W, self.b]

W = np.array([
    [1, -1], [-1, 1], [1, -1], [-1, 1]
], dtype="float32")
b = np.zeros((2,), dtype="float32")
model = TinyDense(W, b)
assert len(model.weights) == 2
```

`self` refers to the instance being initialized or called. Copying the supplied arrays prevents later item changes to the caller's arrays from silently changing this model. The property has no setter, but the arrays inside its returned list are still mutable. A read-only property does not make the objects it exposes immutable.

`features @ W` multiplies a `(5, 4)` matrix by a `(4, 2)` matrix. Adding the `(2,)` bias broadcasts it over rows. Calling `model(features)` invokes the instance's `__call__` method and returns a `(5, 2)` score array.

Worked solution, step 3: slicing batches

PYTHON

```
def take_batch(features, labels, start, batch_size):
    if batch_size <= 0:
        raise ValueError("batch_size must be positive")
    stop = start + batch_size
    return features[start:stop], labels[start:stop]

batch_lengths = []
for start in range(0, len(features), 2):
    x_batch, y_batch = take_batch(features, labels, start, 2)
    scores = model(x_batch)
    batch_lengths.append(len(x_batch))
    assert scores.shape == (len(y_batch), 2)

assert batch_lengths == [2, 2, 1]
```

The three-argument `range(start, stop, step)` produces 0, 2, and 4. Slicing tolerates a stop beyond the end of an array, so the last batch contains one example. A single return statement supplies a tuple of two slices, which the caller unpacks. `.append(...)` mutates the existing list and returns `None`; avoid writing `batch_lengths = batch_lengths.append(...)`.

Worked solution, step 4: evaluate and report

```python
def evaluate(model, features, labels):
    scores = model(features)
    predicted = scores.argmax(axis=1)
    accuracy = float(np.mean(predicted == labels))
    return accuracy, predicted

accuracy, predicted = evaluate(model, features, labels)
print(predicted.tolist())
print(f"Accuracy: {accuracy:.1%}")
assert predicted.tolist() == [1, 0, 1, 0, 0]
assert np.isclose(accuracy, 0.8)
```

The output is `[1, 0, 1, 0, 0]` and `Accuracy: 80.0%`. `.tolist()` converts an array into ordinary Python containers. `float(...)` converts the scalar accuracy into a Python float. The f-string specifier `.1%` formats a fraction as a percentage with one decimal place. `np.isclose` makes a tolerance-based floating-point comparison.

The final all-zero image produces two equal scores. NumPy's `argmax` selects the first maximum, so its predicted class is 0. Its supplied label is 1; therefore exactly four of the five examples match. This lab evaluates raw class scores, so no probability interpretation is required.

Change requests that prove understanding

Change batch size to 3 and predict the batch lengths before running it. Then set `model.b[:] = [0, 0.25]` and predict what happens to the all-zero image. Changing the external `b` alone would not work because the constructor copied it. Finally, remove the copies in `TinyDense.__init__`, mutate `W[0, 0]` after constructing the model, and explain why its behavior changes.

Answers: the lengths become `[3, 2]`; the all-zero image now predicts class 1; without copies, the model and the caller share the same weight array. Recreate the original model afterward if you want the original checkpoints to pass again.

40. A compact Python reference for these chapters

Pattern	Read it as
`from module import name`	Bind this exported name in the current namespace
`f` versus `f(x)`	Function object versus result of calling it
`obj.method(x)`	Call a method associated with this object
`self.x = x`	Save the argument on the instance
`return a, b`	Return one tuple containing two items
`a, b = result`	Unpack two items into two names
`if x is not None:`	Test whether an optional value is present
`for item in items:`	Visit each item in an iterable
`for g, w in zip(gs, ws):`	Pair corresponding gradients and weights
`for _ in range(n):`	Repeat n times, ignoring the loop value
`with context() as name:`	Enter a context, bind its enter-result, then exit
`@decorate` above `def f`	Transform or register a function at definition time
`class Child(Parent):`	Define a class that inherits from another
`super().__init__()`	Run the next initializer in inheritance order
`@property`	Expose a method's returned value through attribute syntax
`x += y`	Request augmented assignment; mutation depends on type
`x @ y`	Matrix multiplication, implemented by the operands' types
`x[i, j]`	Index two array axes in one operation
`x[:, 1:2]`	Keep every row and a one-column slice
`x[:, 1]`	Keep every row, selecting and removing one column axis
`x.shape`, `x.ndim`, `x.dtype`	Axis sizes, number of axes, element representation
`loss:.4f` inside an f-string	Display four decimal places

Before you call a section finished, explain its code aloud without using the phrase "it just does that." Name the object, describe the operation, and state the result's type, shape, or changed state. That is the skill that transfers from these chapters to new Python programs.

Sources and verification

41. What this course is based on

The primary sources are the local third-edition companion notebooks for *Deep Learning with Python* by Francois Chollet and Matthew Watson, together with the repository README and license. All 165 code cells across chapters 2 and 3 were inspected. Explanations are original; source excerpts and adaptations are identified in the lessons. Multiple repetitions of the same Python pattern are consolidated rather than printed repeatedly.

Local notebook basenames:

- `chapter02_mathematical-building-blocks.ipynb`
- `chapter03_introduction-to-ml-frameworks.ipynb`

The repository README identifies the main-folder notebooks as the third edition. Source locators refer to those local files as inspected on 26 October 2026. An upstream notebook may later change, so its cell positions or mistakes may differ.

Source	Purpose
Companion repository	Code provenance and environment guidance
Chapter 2 notebook	Numerical operations, classes, manual training
Chapter 3 notebook	Framework syntax and state-management examples

42. Further reading for a specific question

Use these references when you need the precise contract of an unfamiliar operation. They supplement the local notebook analysis; they are not prerequisites for working through the course.

- Python tutorial: control flow and functions - calls, parameters, loops, and definitions.
- Python tutorial: classes - instances, attributes, methods, and inheritance.
- NumPy indexing - integer indices, slices, and advanced indexing.
- NumPy broadcasting - shape alignment for elementwise operations.
- NumPy copies and views - shared storage and explicit copies.
- NumPy normal sampling - positional `loc` versus keyword `size`.
- IPython custom magics - the backend helper's registration pattern.
- TensorFlow GradientTape - watched variables, constants, and gradient requests.
- TensorFlow function guide - tracing and compiled function behavior.
- PyTorch backward - accumulation into gradient attributes.
- PyTorch no_grad - the update context in the manual step.
- PyTorch functional ReLU - the correct functional activation namespace.
- JAX random numbers - explicit random keys and splitting.
- JAX value_and_grad - scalar objectives, argument selection, and auxiliary outputs.
- JAX JIT compilation - function transformation and tracing.

- Keras layer subclassing - `build`, `call`, and tracked weights.
- Keras training APIs - compile, fit, evaluate, and predict.

43. Verification and code license

The final lab's complete NumPy pipeline was executed, including shape, dtype, batch-boundary, original-data-preservation, prediction, and accuracy assertions. Additional deterministic Python/NumPy examples were run. Every Python code block in the finished course passed a syntax compilation check.

Small TensorFlow/Keras checks were executed locally, including the watched-constant gradient, gradients for Keras variables, optimizer gradient-variable pairing, and activation-resolution repairs. These checks used TensorFlow 2.21.0 and Keras 3.15.1. No full MNIST training run was needed to validate the Python lessons. PyTorch and JAX examples were reviewed against their APIs but were not executed because those packages were not installed in the available local environment. Random outputs and learned metrics are therefore not presented as fixed verified results.

Code from the companion repository is provided under the following license. The course's explanatory prose is newly written.

MIT License

Copyright (c) 2017-present François Chollet

Permission is hereby granted, free of charge, to any person obtaining a copy of this software and associated documentation files (the "Software"), to deal in the Software without restriction, including without limitation the rights to use, copy, modify, merge, publish, distribute, sublicense, and/or sell copies of the Software, and to permit persons to whom the Software is furnished to do so, subject to the following conditions:

The above copyright notice and this permission notice shall be included in all copies or substantial portions of the Software.

THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM, OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN THE SOFTWARE.